HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1

BÁO CÁO BÀI TẬP LỚN

MÔN HỌC: IOT VÀ ỨNG DỤNG

ĐỀ TÀI: HỆ THỐNG QUẢN LÝ THIẾT BỊ IOT

Giảng viên hướng dẫn : **TS. Nguyễn Quốc Uy**
Nhóm học phần : **10**
Sinh viên thực hiện : **Vũ Thùy Dung**
Mã sinh viên : **B23DCCN194**

Hà Nội, 2026

Mục lục

Lời cảm ơn **1**
Chương I. Giới thiệu bài toán **1**
1.1. Bối cảnh 1
1.2. Đối tượng và phạm vi nghiên cứu 1
1.3. Mục tiêu dự án 1
Chương II. Thiết kế tổng quan **3**
2.1. Công nghệ sử dụng 3
2.1.1. Phần cứng 3
2.1.2. Phần mềm 4
2.2. Luồng dữ liệu tổng quan 5
Chương III. Thiết kế chi tiết **6**
3.1. Thiết kế use case 6
3.1.1. Use case Hiển thị biểu đồ dữ liệu cảm biến 6
3.1.2. Use case Hiển thị dữ liệu cảm biến thời gian thực 8
3.1.3. Use case Hiển thị lịch sử dữ liệu cảm biến 10
3.1.4. Use case Hiển thị lịch sử bật/ tắt thiết bị 12
3.1.5. Use case Bật/ tắt thiết bị 14
3.2. Thiết kế cơ sở dữ liệu 16
3.2.1. Mô tả các thực thể và quan hệ 16
3.2.2. Sơ đồ quan hệ ERD 20
3.3. Mô tả API 20
3.3.1. API lấy dữ liệu cảm biến mới nhất 20
3.3.2. API lấy lịch sử dữ liệu cảm biến 21
3.3.3. API lấy danh sách dữ liệu cảm biến 22
3.3.4. API lấy trạng thái thiết bị 23
3.3.5. API điều khiển thiết bị 24
3.3.6. API lấy lịch sử hành động 24
3.3.7. MQTT API - Lấy dữ liệu cảm biến mới nhất 25
3.3.8. MQTT API - Điều khiển thiết bị 26
3.3.9. MQTT API - Lấy trạng thái thiết bị 26
3.3.10. WebSocket API - Lấy dữ liệu cảm biến mới nhất 26
3.3.11. WebSocket API - Lấy trạng thái thiết bị 27
3.4. Thiết kế phần cứng 27
3.4.1. Sơ đồ chân kết nối 27
3.4.2. Đấu nối mạch 28

Chương IV. Kết quả và đánh giá thực nghiệm.....29
4.1. Cấu trúc thư mục dự án..... 29
4.2. Kết quả triển khai phần cứng và MQTT..... 30
4.2.1. Phần cứng.....30
4.2.2. MQTT Broker.....31
4.3. Kết quả triển khai Backend và CSDL..... 31
4.3.1. Backend..... 31
4.3.2. Cơ sở dữ liệu.....32
4.4. Kết quả triển khai Frontend.....32
4.5. Triển khai hệ thống.....35
Chương V. Kết luận..... 36
5.1. Kết quả đạt được.....36
5.2. Hạn chế.....36
5.3. Hướng phát triển.....37

Lời cảm ơn

Lời đầu tiên, em xin gửi lời cảm ơn sâu sắc tới Thầy Nguyễn Quốc Uy, giảng viên phụ trách học phần IoT và Ứng dụng, người đã tận tình hướng dẫn, chỉ bảo và hỗ trợ em trong suốt quá trình nghiên cứu và thực hiện bài tập lớn. Những định hướng và góp ý quý giá của Thầy đã giúp em củng cố kiến thức chuyên môn, nhận ra và khắc phục những hạn chế trong quá trình thực hiện, từ đó hoàn thiện sản phẩm một cách tốt nhất. Em cũng xin trân trọng cảm ơn Thầy đã tạo điều kiện để em có cơ hội triển khai, thử nghiệm và đánh giá thực tế sản phẩm, qua đó tích lũy thêm nhiều kinh nghiệm hữu ích cho bản thân.

Do kiến thức và kinh nghiệm thực tiễn của em còn hạn chế, bài báo cáo này chắc chắn vẫn còn nhiều thiếu sót. Em rất mong nhận được những góp ý, nhận xét từ Thầy để có thể tiếp tục hoàn thiện và nâng cao năng lực của mình trong các dự án sau này.

Chương I. Giới thiệu bài toán

1.1. Bối cảnh

Trong những năm gần đây, sự phát triển của Internet vạn vật (Internet of Things – IoT) đã tạo ra nhiều cơ hội ứng dụng công nghệ vào hoạt động giám sát, quản lý và điều khiển thiết bị từ xa. IoT cho phép các thiết bị phần cứng thu thập dữ liệu từ môi trường, truyền thông tin qua mạng và phối hợp với các hệ thống phần mềm để cung cấp khả năng giám sát và điều khiển thuận tiện hơn.

Trong thực tế, việc theo dõi các thông số môi trường như nhiệt độ, độ ẩm và cường độ ánh sáng thường đòi hỏi người dùng phải quan sát trực tiếp hoặc kiểm tra thiết bị tại vị trí lắp đặt. Bên cạnh đó, việc điều khiển các thiết bị điện theo phương thức thủ công có thể gây bất tiện, đặc biệt khi người dùng cần theo dõi và quản lý nhiều thiết bị cùng lúc. Do đó, việc xây dựng một hệ thống có khả năng tự động thu thập dữ liệu, hiển thị thông tin và điều khiển thiết bị thông qua giao diện Web là một hướng ứng dụng có tính thực tiễn.

Xuất phát từ nhu cầu trên, đề tài **"Hệ thống quản lý thiết bị IoT"** được thực hiện nhằm xây dựng một mô hình IoT có khả năng thu thập dữ liệu cảm biến, truyền dữ liệu qua giao thức MQTT, xử lý và lưu trữ thông tin tại máy chủ, đồng thời cho phép người dùng theo dõi và điều khiển thiết bị thông qua trình duyệt Web.

1.2. Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu: Đề tài tập trung nghiên cứu và xây dựng hệ thống IoT phục vụ giám sát các thông số môi trường và điều khiển thiết bị thông qua giao diện Web. Nội dung nghiên cứu bao gồm việc thu thập, truyền tải, xử lý và lưu trữ dữ liệu cảm biến; cơ chế giao tiếp giữa thiết bị phần cứng và máy chủ; cũng như phương thức hiển thị dữ liệu và điều khiển thiết bị trên nền tảng Web.

Phạm vi nghiên cứu: Đề tài được triển khai ở quy mô mô hình thử nghiệm, sử dụng ESP32 kết hợp với các cảm biến nhiệt độ, độ ẩm, ánh sáng và các thiết bị LED. Hệ thống sử dụng giao thức MQTT để trao đổi dữ liệu, Spring Boot để xây dựng Backend, MySQL để lưu trữ dữ liệu và HTML, CSS, JavaScript để phát triển giao diện Web. Phạm vi thực hiện tập trung vào các chức năng giám sát dữ liệu cảm biến, điều khiển bật/tắt thiết bị, cập nhật trạng thái theo thời gian thực và tra cứu lịch sử hoạt động trong môi trường mạng cục bộ.

1.3. Mục tiêu dự án

Mục tiêu của dự án là xây dựng hệ thống IoT giám sát và điều khiển thiết bị thông qua giao diện Web, cho phép người dùng theo dõi các thông số môi trường và thực hiện điều khiển thiết bị từ máy tính thông qua mạng. Hệ thống hướng đến việc kết hợp phần cứng, giao thức truyền thông, Backend, cơ sở dữ liệu và giao diện Web thành một mô hình hoạt

động thống nhất.

Cụ thể, dự án hướng đến các mục tiêu sau:

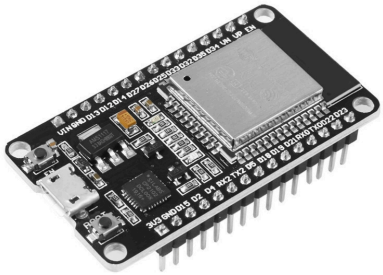
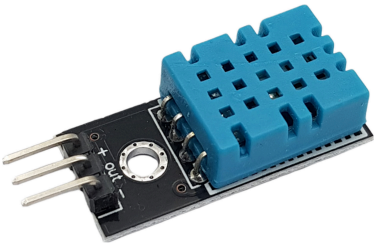
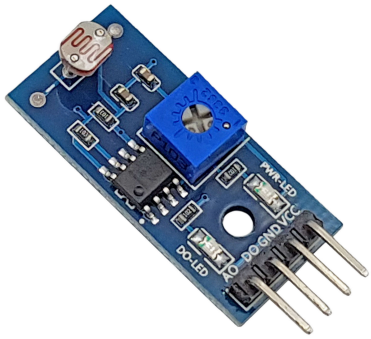
- Xây dựng mô hình phần cứng sử dụng ESP32 và các cảm biến để thu thập dữ liệu nhiệt độ, độ ẩm và ánh sáng.
- Ứng dụng giao thức MQTT để truyền dữ liệu cảm biến, gửi lệnh điều khiển và nhận trạng thái phản hồi từ thiết bị.
- Phát triển Backend bằng Spring Boot để xử lý dữ liệu, cung cấp REST API và hỗ trợ cập nhật thông tin theo thời gian thực thông qua WebSocket.
- Xây dựng cơ sở dữ liệu MySQL để lưu trữ dữ liệu cảm biến, thông tin thiết bị và lịch sử điều khiển.
- Thiết kế giao diện Web trực quan, hỗ trợ giám sát dữ liệu, bật/tắt thiết bị và tra cứu lịch sử hoạt động.
- Kiểm thử hoạt động của hệ thống, đánh giá khả năng trao đổi dữ liệu giữa các thành phần và đề xuất hướng cải tiến nhằm nâng cao tính ổn định, bảo mật và khả năng mở rộng.

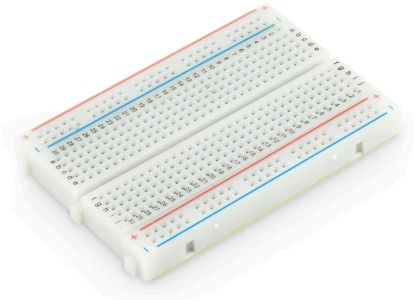
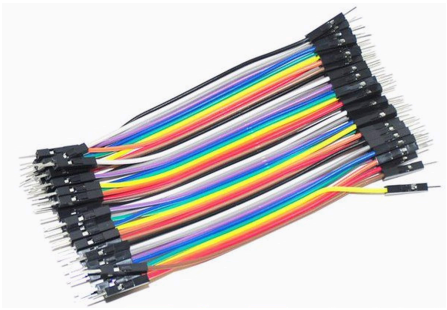
Thông qua dự án, vận dụng kiến thức về IoT, lập trình nhúng, phát triển Web, cơ sở dữ liệu và truyền thông mạng để xây dựng một hệ thống giám sát, điều khiển thiết bị có tính ứng dụng thực tiễn và làm nền tảng cho các hệ thống IoT mở rộng trong tương lai.

Chương II. Thiết kế tổng quan

2.1. Công nghệ sử dụng

2.1.1. Phần cứng

Thiết bị	Hình ảnh minh họa
Mạch điều khiển ESP32	
Cảm biến nhiệt độ, độ ẩm DHT11	
Cảm biến quang trở	
LED	

Breadboard	
Dây nối	

Bảng 2.1. Các thiết bị phần cứng được sử dụng

2.1.2. Phần mềm

a. Front-end: HTML, CSS, JavaScript

Trong xây dựng Front-end, ba công cụ nền tảng không thể thiếu là HTML, CSS và JavaScript.

- HTML: Dùng để tạo cấu trúc và nội dung cho trang web, như văn bản, hình ảnh, và liên kết.
- CSS: Dùng để thiết kế và định kiểu giao diện, như màu sắc, bố cục, và hiệu ứng.
- JavaScript: đảm nhận phần hành vi, mang đến sự tương tác và động cho trang web, ví dụ như xử lý sự kiện, cập nhật dữ liệu theo thời gian thực hay điều khiển logic phía client.

b. Back-end: Node.js

- Node.js là một môi trường chạy JavaScript phía server dựa trên công cụ V8 JavaScript của Google. Với Node.js, bạn có thể xây dựng các ứng dụng web, API, công cụ CLI, và nhiều loại ứng dụng khác mà không cần phải dựa vào trình duyệt.
- Các đặc điểm nổi bật:
  - Asynchronous & Non-blocking I/O: Node.js xử lý nhiều yêu cầu cùng lúc mà không chặn luồng chính.
  - Event-driven: Dựa trên cơ chế vòng lặp sự kiện (event loop), rất hiệu quả với các ứng dụng thời gian thực.
  - Single-threaded: Mặc dù chỉ dùng một luồng chính, Node.js có thể xử lý rất nhiều kết nối đồng thời nhờ kiến trúc non-blocking.
  - Rich Ecosystem: Có thư viện npm (Node Package Manager) khổng lồ.

c. Cơ sở dữ liệu: MySQL

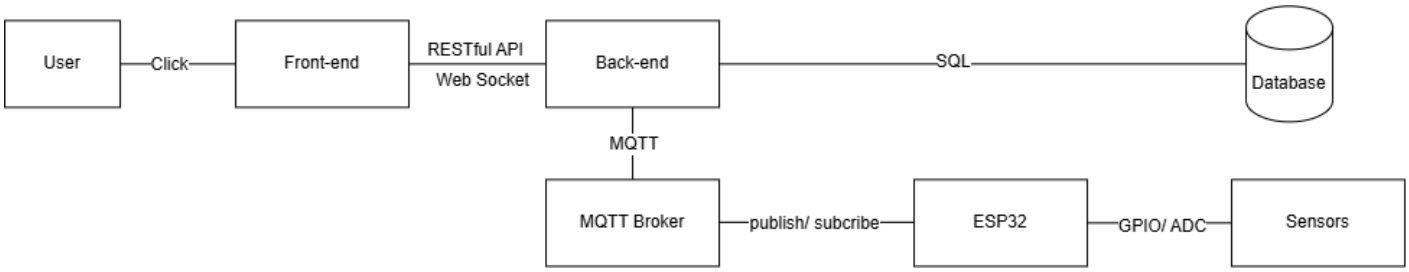
Xây dựng cơ sở dữ liệu bằng MySQL giúp lưu trữ và quản lý dữ liệu có cấu trúc an toàn thông qua ngôn ngữ SQL chuẩn. Với mô hình quan hệ chặt chẽ, hỗ trợ giao dịch chuẩn ACID và tốc độ truy vấn cao, MySQL là giải pháp tối ưu cho việc phát triển các ứng dụng web và hệ thống thông tin linh hoạt.

d. MQTT broker: Mosquitto

- Mosquitto là một phần mềm mã nguồn mở cung cấp MQTT (Message Queuing Telemetry Transport) - một giao thức truyền tải tin nhắn nhẹ, được thiết kế cho các thiết bị IoT và ứng dụng cần truyền dữ liệu hiệu quả qua mạng. Mosquitto có vai trò như một MQTT broker (máy chủ trung gian), cho phép các client (thiết bị hoặc ứng dụng) xuất bản (publish) và đăng ký (subscribe) tin nhắn.
- Tính năng nổi bật của Mosquitto:
  - Nhẹ và hiệu quả: Phù hợp với các thiết bị IoT hạn chế tài nguyên.
  - Hỗ trợ chuẩn MQTT 3.1 và 3.1.1: Tương thích với nhiều ứng dụng IoT.
  - Mã nguồn mở:Có thể tùy chỉnh để phù hợp với nhu cầu.
  - Hỗ trợ bảo mật:
    + Kết nối an toàn bằng TLS.
    + Xác thực qua username/password hoặc chứng chỉ.
  - Đa nền tảng: Có thể chạy trên Windows, Linux, macOS và các thiết bị nhúng.
- Ứng dụng của Mosquitto trong IoT:
  - Giám sát và điều khiển thiết bị từ xa.
  - Truyền tải dữ liệu cảm biến đến server hoặc các ứng dụng.
  - Kết nối nhiều thiết bị IoT với chi phí thấp.

2.2. Luồng dữ liệu tổng quan

Người dùng thao tác trên giao diện. Front-end gửi request đến Back-end thông qua RESTful API. Back-end xử lý truy vấn/ lưu dữ liệu với Database MySQL, đồng thời giao tiếp với ESP32 bằng giao thức MQTT thông qua Mosquitto Broker. ESP32 đọc dữ liệu từ các cảm biến thông qua GPIO/ ADC và gửi dữ liệu lên MQTT Broker. WebSocket được sử dụng để Back-end đẩy dữ liệu cảm biến và trạng thái thiết bị theo thời gian thực mà không cần người dùng tải lại trang.



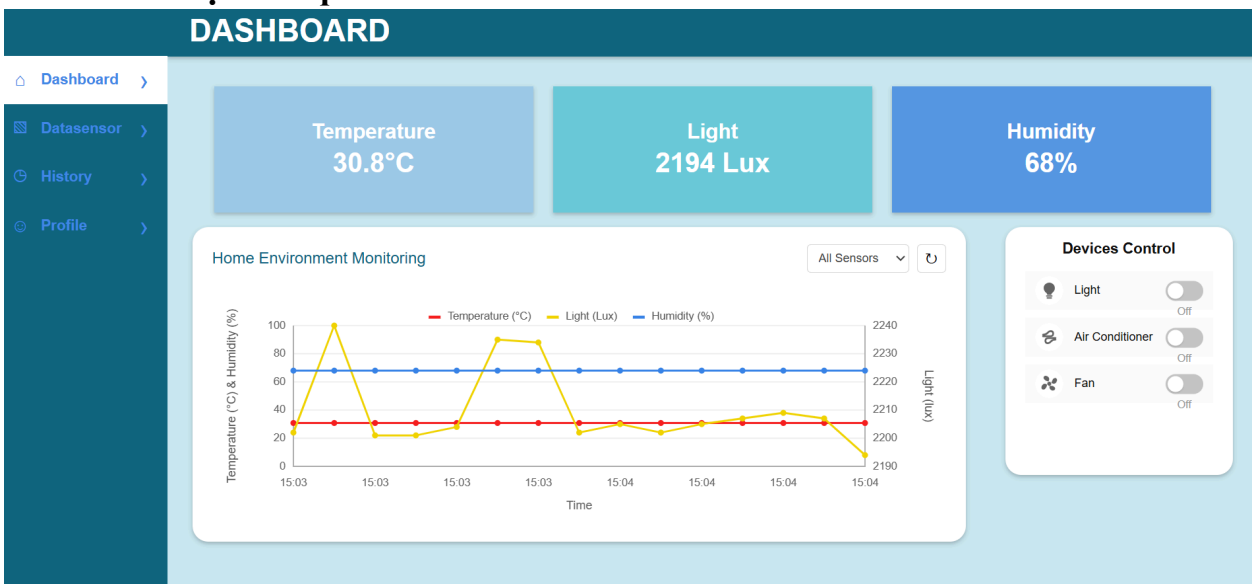
Hình 2.1. Luồng dữ liệu tổng quan

Chương III. Thiết kế chi tiết

3.1. Thiết kế use case

3.1.1. Use case Hiển thị biểu đồ dữ liệu cảm biến

a. Giao diện liên quan

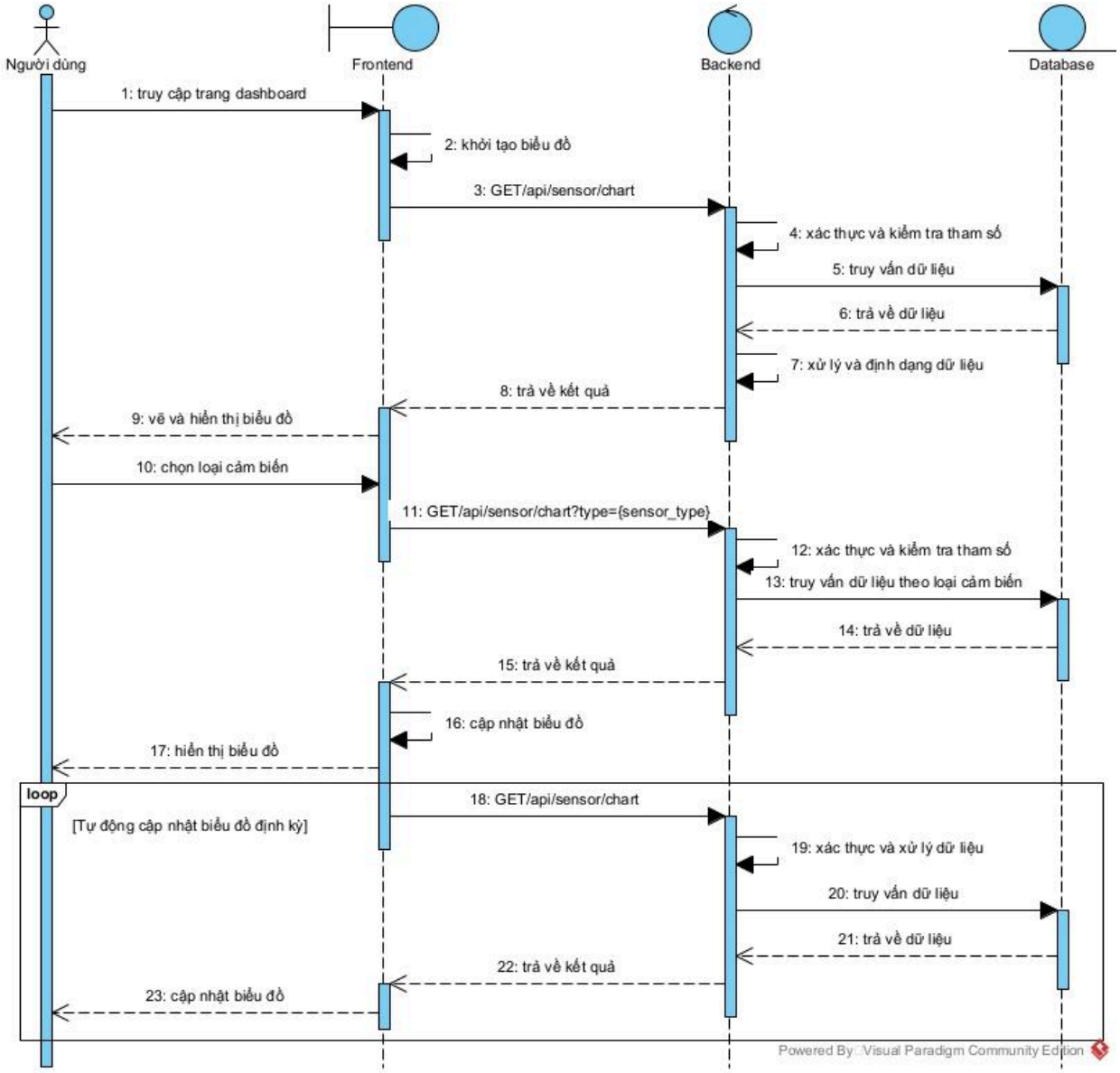


Hình 3.1. Giao diện biểu đồ dữ liệu cảm biến thời gian thực trong trang Dashboard

b. Đặc tả use case

- Actor: Người dùng.
- Mục đích: Cho phép người dùng xem sự thay đổi của giá trị cảm biến theo thời gian dưới dạng biểu đồ.
- Tiền điều kiện:
  + Người dùng đã truy cập vào trang Dashboard.
  + Frontend đã kết nối được với Backend.
  + Cơ sở dữ liệu có dữ liệu cảm biến.
- Hậu điều kiện:
  + Biểu đồ được hiển thị trên giao diện.
  + Các giá trị cảm biến gần nhất được thể hiện theo thời gian.

c. Sequence diagram



Hình 3.2. Sơ đồ tuần tự usecase Hiển thị biểu đồ dữ liệu cảm biến

- Luồng khởi tạo và hiển thị biểu đồ:
- Bước 1: Người dùng truy cập trang Dashboard.
- Bước 2: Frontend khởi tạo khu vực biểu đồ.
- Bước 3: Frontend gửi request GET /api/sensor/chart đến Backend để lấy dữ liệu biểu đồ.
- Bước 4: Backend xác thực request và kiểm tra các tham số.
- Bước 5: Backend truy vấn dữ liệu cảm biến từ Database.
- Bước 6: Database trả dữ liệu cảm biến về cho Backend.
- Bước 7: Backend xử lý và định dạng dữ liệu phù hợp để hiển thị biểu đồ.
- Bước 8: Backend trả kết quả dữ liệu về cho Frontend.
- Bước 9: Frontend nhận dữ liệu và vẽ, hiển thị biểu đồ cho người dùng.
- Luồng lọc biểu đồ theo loại cảm biến:

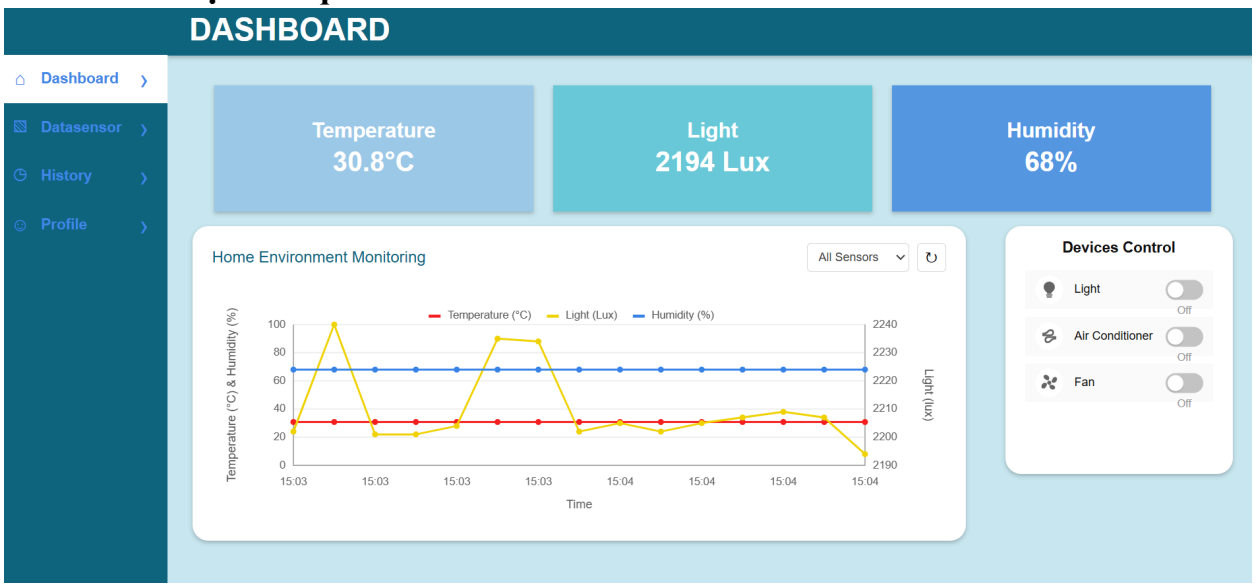
- Bước 10: Người dùng chọn loại cảm biến trên giao diện.
- Bước 11: Frontend gửi request GET /api/sensor/chart?type={sensor_type} đến Backend.
- Bước 12: Backend xác thực request và kiểm tra tham số loại cảm biến.
- Bước 13: Backend truy vấn Database để lấy dữ liệu thuộc loại cảm biến được lựa chọn.
- Bước 14: Database trả dữ liệu về cho Backend.
- Bước 15: Backend xử lý và trả kết quả về cho Frontend.
- Bước 16: Frontend cập nhật dữ liệu và render lại biểu đồ.
- Bước 17: Biểu đồ mới được hiển thị cho người dùng.

● Luồng tự động cập nhật biểu đồ:

- Bước 18: Frontend định kỳ gửi request GET /api/sensor/chart đến Backend.
- Bước 19: Backend tiếp nhận request và xác thực, xử lý yêu cầu.
- Bước 20: Backend truy vấn dữ liệu cảm biến mới nhất từ Database.
- Bước 21: Database trả dữ liệu về cho Backend.
- Bước 22: Backend xử lý và trả kết quả dữ liệu về cho Frontend.
- Bước 23: Frontend nhận dữ liệu và cập nhật lại biểu đồ.

3.1.2. Use case Hiển thị dữ liệu cảm biến thời gian thực

a. Giao diện liên quan



Hình 3.3. Giao diện gồm 3 thẻ dữ liệu cảm biến thời gian thực trong trang Dashboard

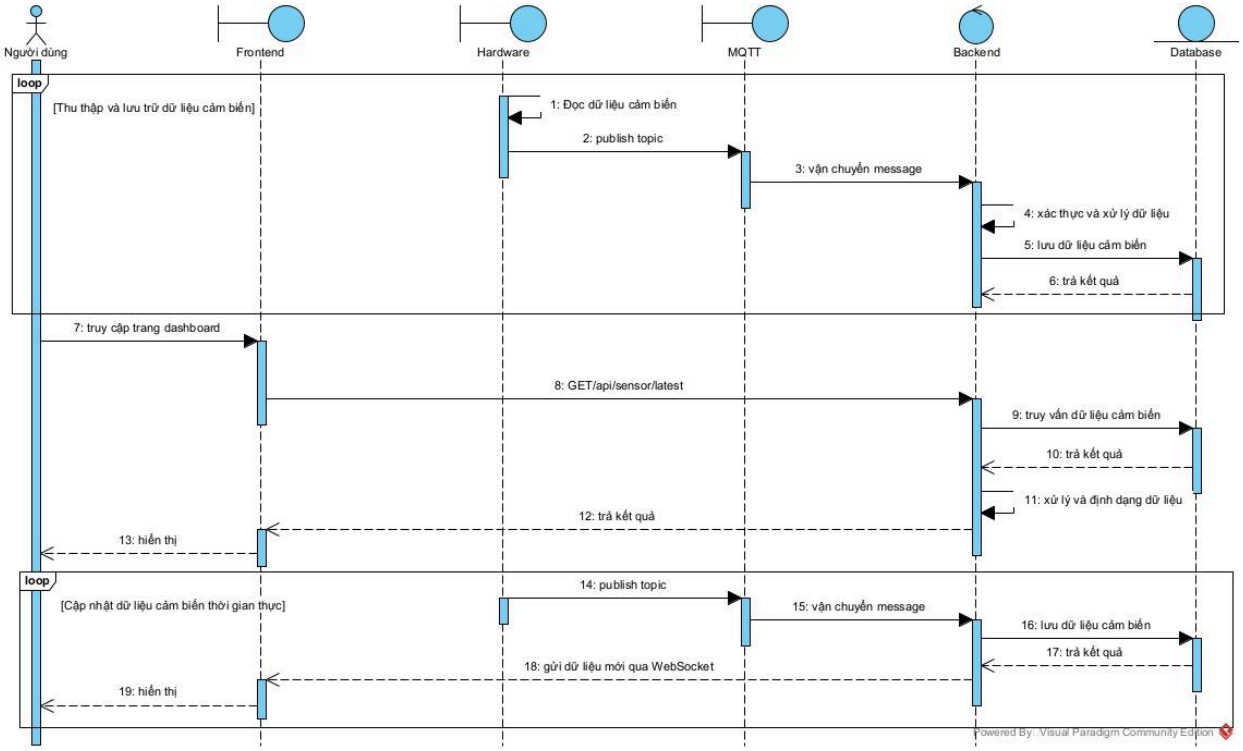
b. Đặc tả use case

- Actor: Người dùng.
- Mục đích: Cho phép người dùng theo dõi các giá trị cảm biến mới nhất trên Dashboard mà không cần tải lại trang.
- Tiền điều kiện:
  + Người dùng đã mở Dashboard.
  + Frontend kết nối được với Backend.
  + ESP32 đang thu thập dữ liệu cảm biến.

+ MQTT Broker đang hoạt động.

- Hậu điều kiện:
  + Các giá trị cảm biến mới nhất được cập nhật trên giao diện.
  + Giao diện được cập nhật tự động khi có dữ liệu mới.

c. Sequence diagram



Hình 3.4. Sơ đồ tuần tự usecase Hiển thị dữ liệu cảm biến thời gian thực

- Luồng thu thập và lưu trữ dữ liệu cảm biến (diễn ra tự động và liên tục):
- Bước 1: Hardware (ESP32) đọc dữ liệu từ các cảm biến như nhiệt độ, độ ẩm, ánh sáng.
- Bước 2: ESP32 publish dữ liệu cảm biến lên MQTT Broker thông qua topic tương ứng.
- Bước 3: MQTT Broker nhận và chuyển tiếp message đến Backend Node.js do Backend đã subscribe topic dữ liệu cảm biến.
- Bước 4: Backend tiếp nhận, xác thực và xử lý dữ liệu cảm biến.
- Bước 5: Backend lưu dữ liệu cảm biến vào Database MySQL.
- Bước 6: Database trả kết quả xác nhận việc lưu dữ liệu cho Backend.
- Luồng người dùng truy cập trang Dashboard:
- Bước 7: Người dùng truy cập trang Dashboard.
- Bước 8: Frontend gửi request GET /api/sensor/latest đến Backend để lấy dữ liệu cảm biến mới nhất.
- Bước 9: Backend truy vấn Database để lấy các giá trị cảm biến mới nhất.
- Bước 10: Database trả dữ liệu cảm biến về cho Backend.
- Bước 11: Backend xử lý và định dạng dữ liệu thành JSON phù hợp với Frontend.

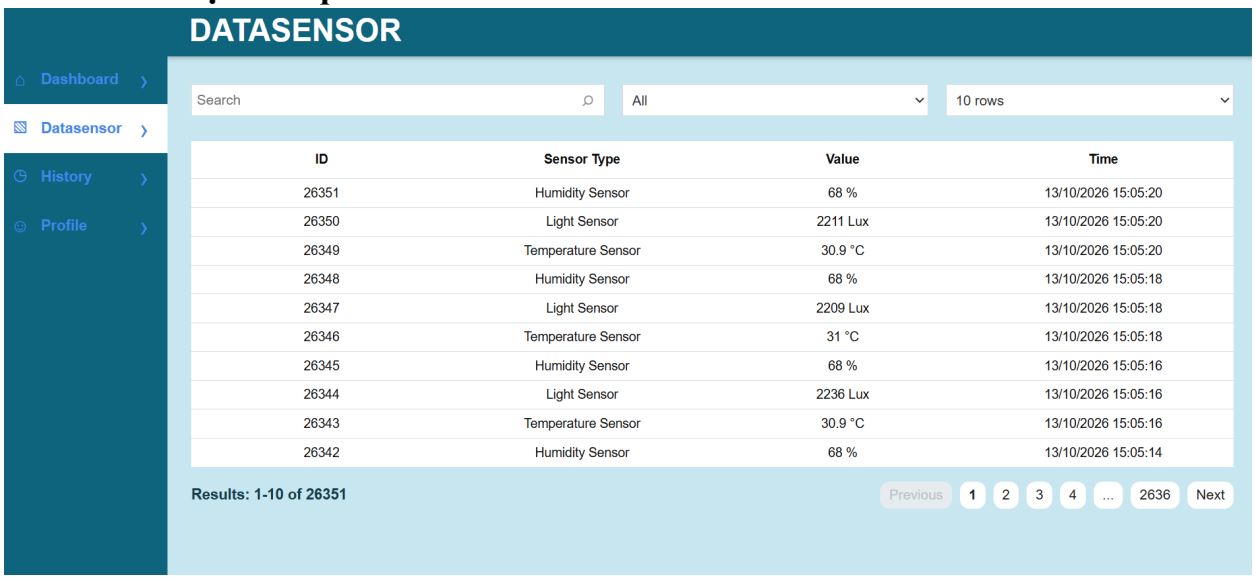
- Bước 12: Backend trả kết quả JSON về cho Frontend.
- Bước 13: Frontend nhận dữ liệu và hiển thị các giá trị cảm biến trên Dashboard cho người dùng.

● Luồng cập nhật dữ liệu cảm biến thời gian thực:

- Bước 14: ESP32 đọc dữ liệu cảm biến mới và publish dữ liệu lên MQTT Broker.
- Bước 15: MQTT Broker nhận và chuyển tiếp message đến Backend.
- Bước 16: Backend xử lý dữ liệu cảm biến và lưu dữ liệu mới vào Database MySQL.
- Bước 17: Database trả kết quả xác nhận việc lưu dữ liệu cho Backend.
- Bước 18: Backend gửi dữ liệu cảm biến mới đến Frontend thông qua WebSocket.
- Bước 19: Frontend nhận dữ liệu và cập nhật giá trị cảm biến trên giao diện Dashboard theo thời gian thực.

3.1.3. Use case Hiển thị lịch sử dữ liệu cảm biến

a. Giao diện liên quan

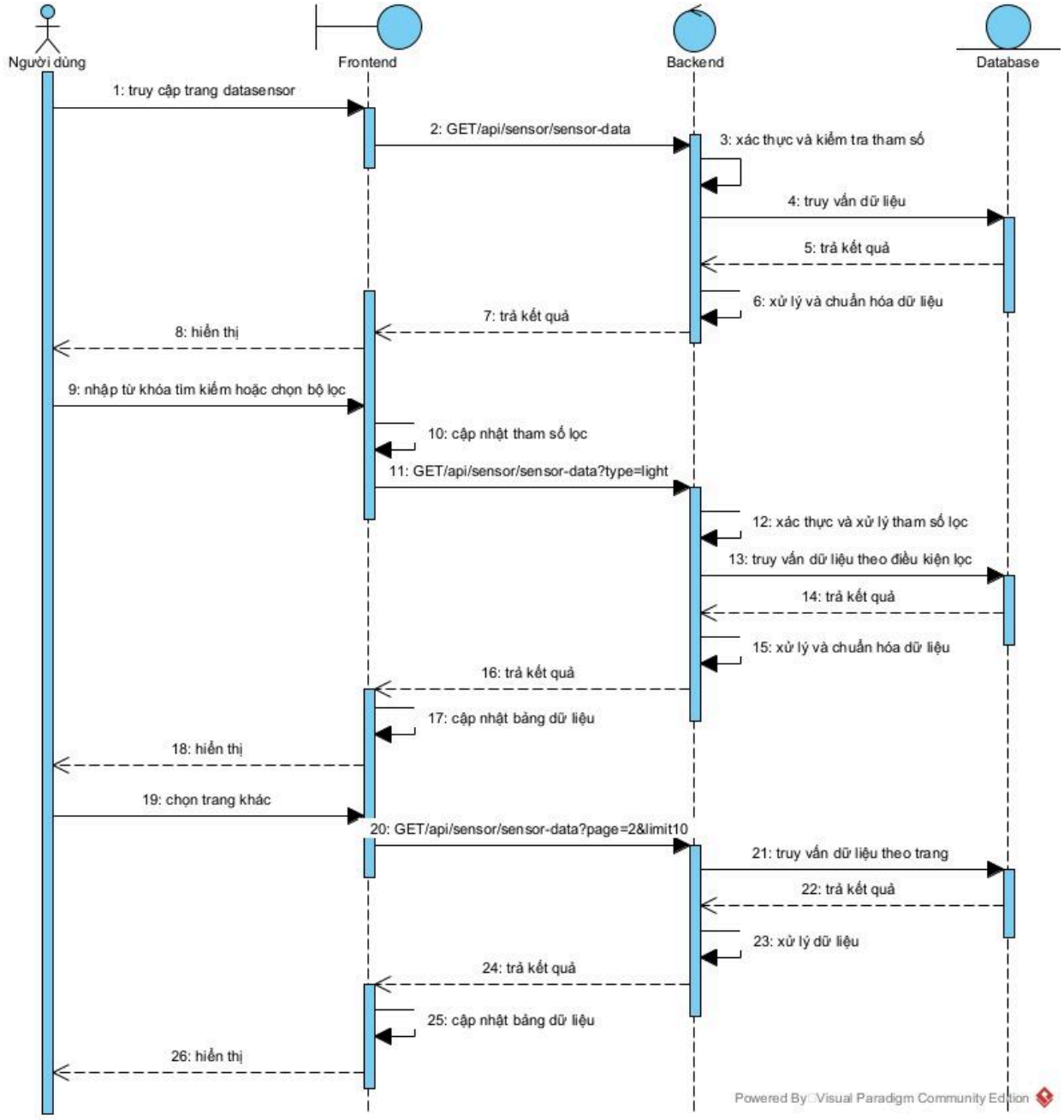


Hình 3.5. Giao diện hiển thị Dữ liệu cảm biến (Trang Datasensor)

b. Đặc tả use case

- Actor: Người dùng.
- Mục đích: Cho phép người dùng xem lại các dữ liệu cảm biến đã được hệ thống thu thập.
- Tiền điều kiện:
  + Người dùng truy cập trang Data Sensor.
  + Database có dữ liệu cảm biến.
- Hậu điều kiện:
  + Danh sách dữ liệu cảm biến được hiển thị trên giao diện.
  + Người dùng có thể tìm kiếm, lọc và phân trang dữ liệu.

c. Sequence diagram



Hình 3.6. Sơ đồ tuần tự usecase Hiển thị lịch sử dữ liệu cảm biến

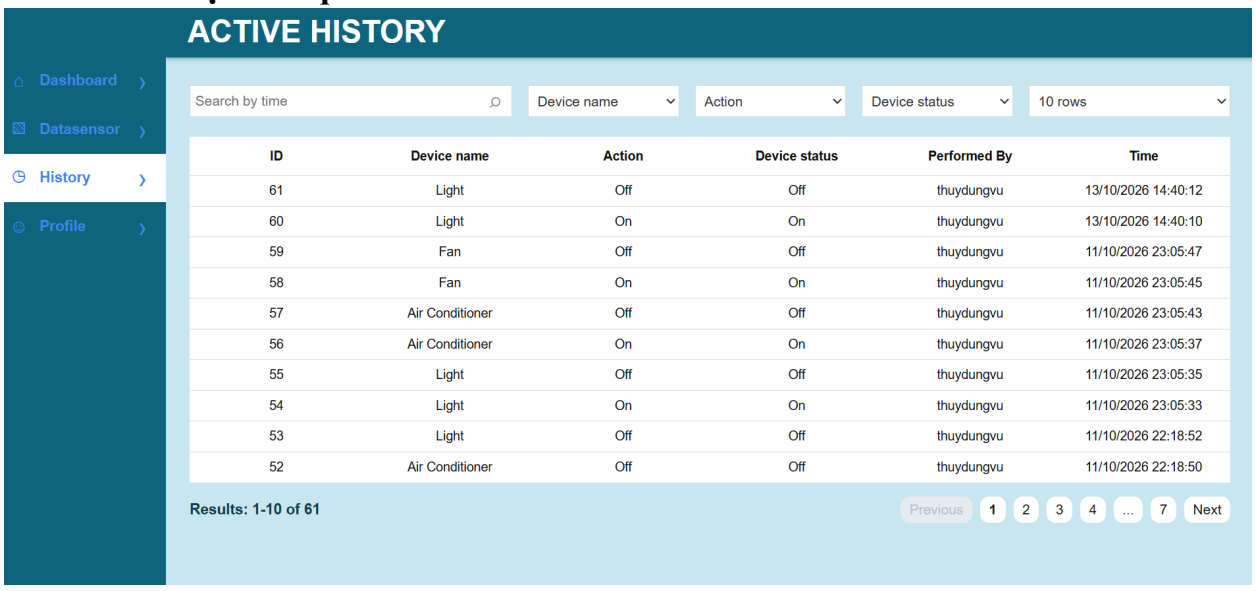
- Luồng người dùng truy cập trang Datasensor:

- Bước 1: Người dùng truy cập trang datasensor.
- Bước 2: Frontend gửi request GET /api/sensor/sensor-data đến Backend.
- Bước 3: Backend xác thực và kiểm tra tham số của request.
- Bước 4: Backend truy vấn dữ liệu cảm biến từ Database.
- Bước 5: Database trả về kết quả truy vấn cho Backend.
- Bước 6: Backend xử lý và chuẩn hóa dữ liệu thành JSON.
- Bước 7: Backend trả kết quả về cho Frontend.
- Bước 8: Frontend hiển thị dữ liệu cảm biến trên bảng cho người dùng.

- Luồng người dùng tìm kiếm hoặc lọc dữ liệu cảm biến:
- Bước 9: Người dùng nhập từ khóa tìm kiếm hoặc chọn loại cảm biến cần lọc.
- Bước 10: Frontend cập nhật tham số lọc.
- Bước 11: Frontend gửi request đến Backend, ví dụ: GET /api/sensor/sensor-data?type=light.
- Bước 12: Backend xác thực và xử lý các tham số lọc.
- Bước 13: Backend truy vấn Database theo điều kiện lọc.
- Bước 14: Database trả về kết quả truy vấn.
- Bước 15: Backend xử lý và chuẩn hóa dữ liệu.
- Bước 16: Backend trả kết quả về cho Frontend.
- Bước 17: Frontend cập nhật lại bảng dữ liệu theo kết quả tìm kiếm/lọc.
- Bước 18: Người dùng xem kết quả dữ liệu đã được lọc.
- Luồng người dùng phân trang dữ liệu:
- Bước 19: Người dùng chọn một trang khác trong bảng dữ liệu.
- Bước 20: Frontend gửi request đến Backend kèm thông tin phân trang, ví dụ: GET /api/sensor/sensor-data?page=2&limit=10.
- Bước 21: Backend truy vấn Database theo trang được yêu cầu.
- Bước 22: Database trả về dữ liệu tương ứng với trang đó.
- Bước 23: Backend xử lý dữ liệu và chuẩn hóa kết quả.
- Bước 24: Backend trả kết quả về cho Frontend.
- Bước 25: Frontend cập nhật lại bảng dữ liệu.
- Bước 26: Người dùng xem dữ liệu ở trang được chọn.

3.1.4. Use case Hiển thị lịch sử bật/ tắt thiết bị

a. Giao diện liên quan



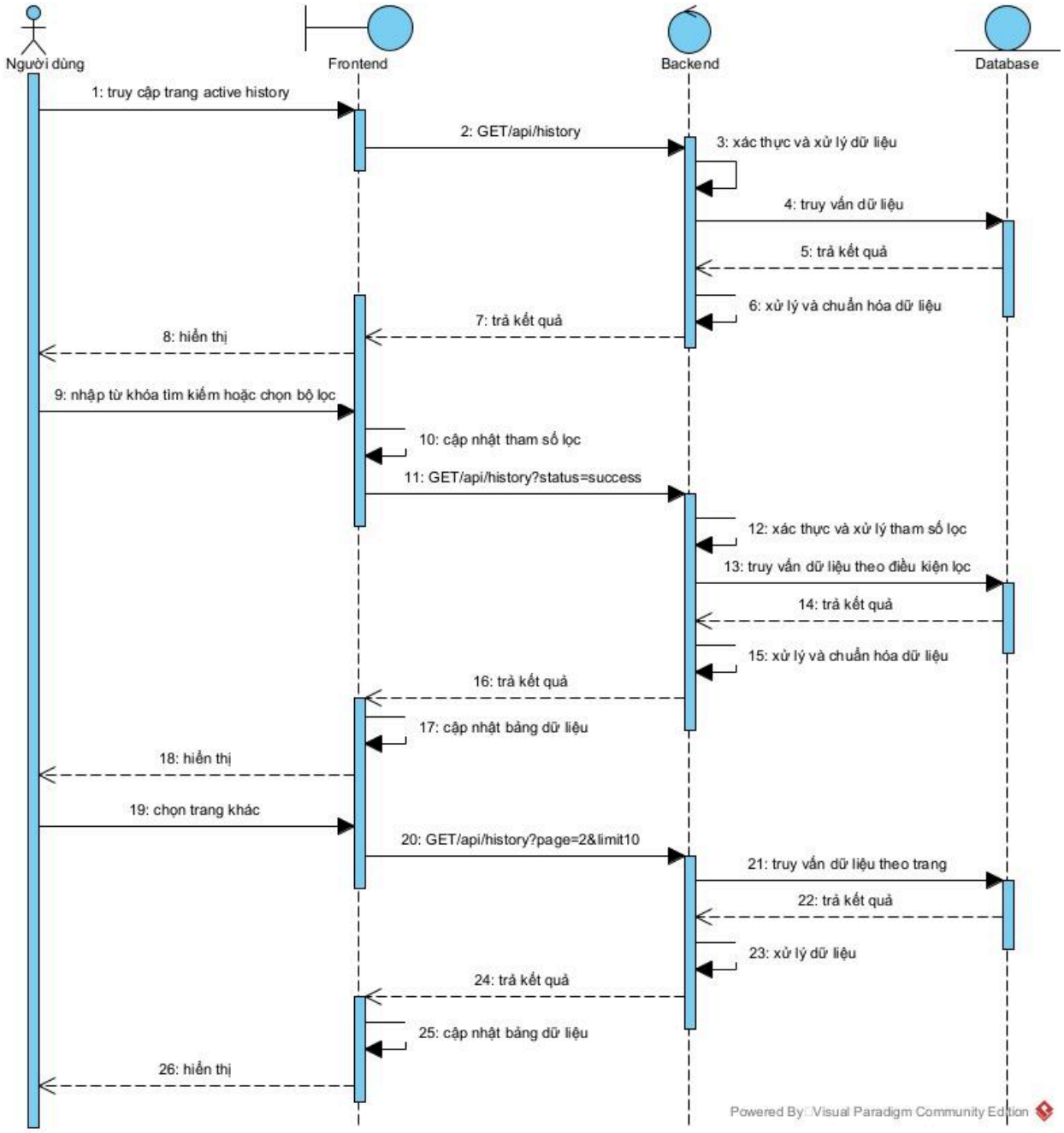
Hình 3.7. Giao diện hiển thị lịch sử bật/ tắt thiết bị (Trang Active History)

b. Đặc tả use case

- Actor: Người dùng
- Mục đích: Cho phép người dùng xem lại các thao tác bật/ tắt thiết bị đã thực hiện.

- Tiền điều kiện:
    + Người dùng đã truy cập trang History.
    + Database có dữ liệu lịch sử.
- Hậu điều kiện:
    + Danh sách lịch sử điều khiển thiết bị được hiển thị.
    + Các bản ghi được sắp xếp theo thời gian giảm dần.

c. Sequence diagram

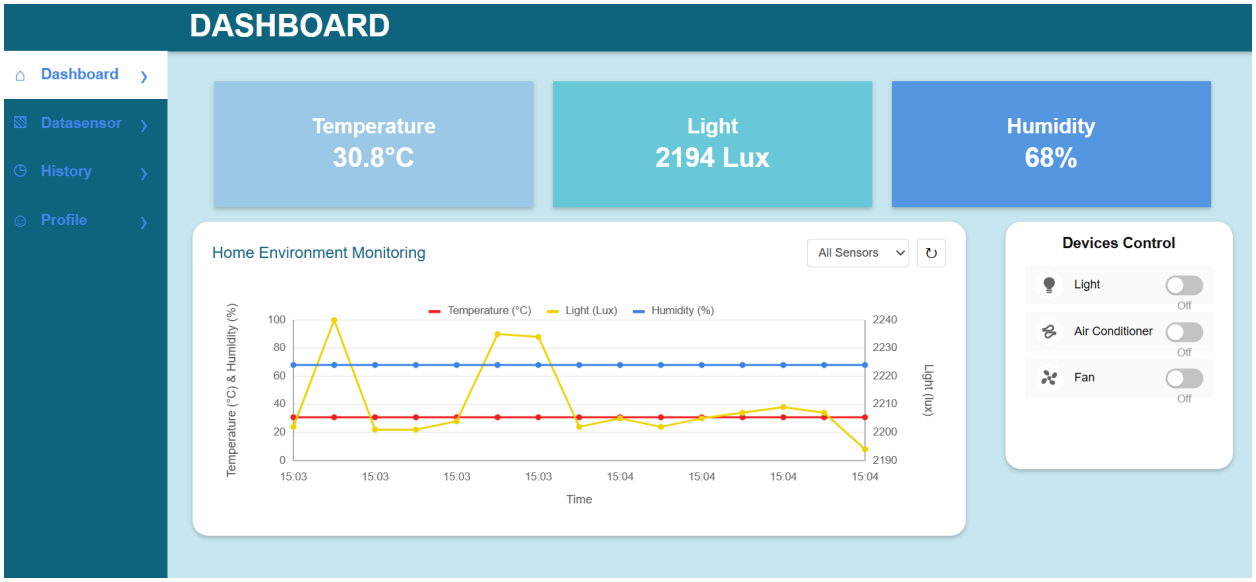


Hình 3.8. Sơ đồ tuần tự usecase Hiển thị lịch sử bật/ tắt thiết bị

- Luồng người dùng truy cập trang Active History:
- Bước 1: Người dùng truy cập trang Active History.
- Bước 2: Frontend gửi request GET /api/history đến Backend.
- Bước 3: Backend xác thực và kiểm tra tham số của request.
- Bước 4: Backend truy vấn lịch sử hoạt động của thiết bị từ Database.
- Bước 5: Database trả về kết quả truy vấn cho Backend.
- Bước 6: Backend xử lý và chuẩn hóa dữ liệu thành JSON.
- Bước 7: Backend trả kết quả về cho Frontend.
- Bước 8: Frontend hiển thị lịch sử hoạt động trên bảng cho người dùng.
- Luồng người dùng tìm kiếm lịch sử hoạt động:
- Bước 9: Người dùng nhập từ khóa tìm kiếm.
- Bước 10: Frontend cập nhật tham số lọc.
- Bước 11: Frontend gửi request đến Backend, ví dụ: GET /api/history?status=success.
- Bước 12: Backend xác thực và xử lý các tham số lọc.
- Bước 13: Backend truy vấn Database theo điều kiện lọc.
- Bước 14: Database trả về kết quả truy vấn.
- Bước 15: Backend xử lý và chuẩn hóa dữ liệu.
- Bước 16: Backend trả kết quả về cho Frontend.
- Bước 17: Frontend cập nhật lại bảng dữ liệu theo kết quả tìm kiếm/lọc.
- Bước 18: Người dùng xem kết quả dữ liệu đã được lọc.
- Luồng người dùng phân trang dữ liệu:
- Bước 19: Người dùng chọn một trang khác trong bảng dữ liệu.
- Bước 20: Frontend gửi request đến Backend kèm thông tin phân trang, ví dụ: GET /api/history?page=2&limit=10.
- Bước 21: Backend truy vấn Database theo trang được yêu cầu.
- Bước 22: Database trả về dữ liệu tương ứng với trang đó.
- Bước 23: Backend xử lý dữ liệu và chuẩn hóa kết quả.
- Bước 24: Backend trả kết quả về cho Frontend.
- Bước 25: Frontend cập nhật lại bảng dữ liệu.
- Bước 26: Người dùng xem dữ liệu ở trang được chọn.

3.1.5. Use case Bật/ tắt thiết bị

a. Giao diện liên quan

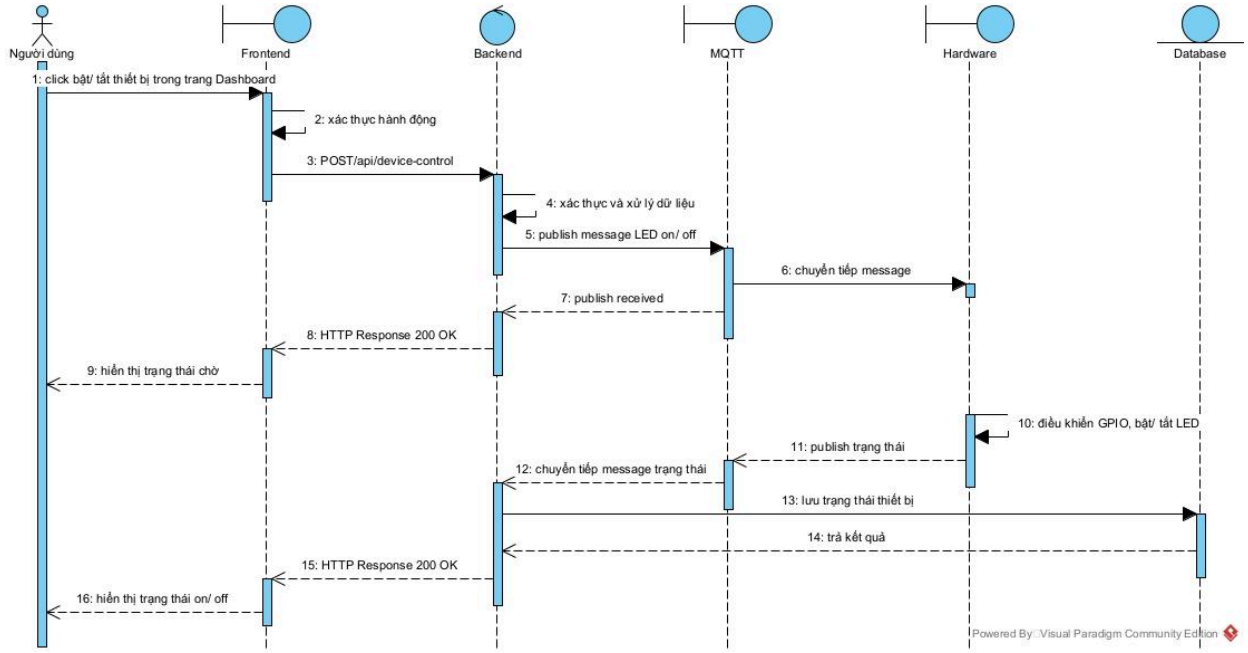


Hình 3.9. Giao diện gồm các nút điều khiển thiết bị trong Devices Control của trang Dashboard

b. Đặc tả use case

- Actor: Người dùng
- Mục đích: Cho phép người dùng điều khiển trạng thái thiết bị thông qua giao diện web.
- Tiền điều kiện:
  + Người dùng đang truy cập Dashboard.
  + Frontend kết nối được với Backend.
  + Backend kết nối được với MQTT Broker.
  + ESP32 đang hoạt động và subscribe MQTT topic điều khiển.
- Hậu điều kiện:
  + Thiết bị được bật hoặc tắt.
  + Trạng thái thiết bị được cập nhật.
  + Thao tác được lưu vào bảng Action_history.

c. Sequence diagram



Hình 3.10. Sơ đồ tuần tự usecase Bật/ tắt thiết bị

- Luồng khởi tạo lệnh điều khiển:
- Bước 1: Người dùng click nút bật/tắt thiết bị trên trang Dashboard.
- Bước 2: Frontend xác thực hành động và xác định trạng thái cần thay đổi.
- Bước 3: Frontend gửi yêu cầu POST /api/device-control đến Backend.
- Bước 4: Backend xác thực và xử lý dữ liệu yêu cầu.
- Bước 5: Backend publish lệnh bật/tắt LED lên MQTT Broker.
- Bước 6: MQTT Broker chuyển tiếp message điều khiển đến ESP32.
- Bước 7: MQTT Broker gửi phản hồi xác nhận đã nhận message về Backend.
- Bước 8: Backend trả HTTP Response 200 OK cho Frontend.
- Luồng chờ phản hồi và thực thi lệnh:
- Bước 9: Frontend hiển thị trạng thái chờ xử lý cho người dùng.
- Bước 10: ESP32 nhận lệnh và điều khiển GPIO để bật hoặc tắt LED.
- Bước 11: ESP32 publish trạng thái thực tế của thiết bị lên MQTT Broker.
- Bước 12: MQTT Broker chuyển tiếp message trạng thái đến Backend.
- Bước 13: Backend lưu trạng thái thiết bị vào Database.
- Bước 14: Database trả kết quả lưu dữ liệu về Backend.
- Bước 15: Backend trả HTTP Response 200 OK về Frontend sau khi xử lý trạng thái thiết bị.
- Bước 16: Frontend cập nhật và hiển thị trạng thái bật/tắt thực tế của thiết bị cho người dùng.

3.2. Thiết kế cơ sở dữ liệu

3.2.1. Mô tả các thực thể và quan hệ

a. Các thực thể

- Thực thể **Users**: Lưu trữ thông tin người dùng sử dụng hệ thống. Mỗi người dùng được xác định duy nhất bởi id và có các thông tin như tên đăng nhập, mật khẩu đã mã hóa, họ tên và email. Thực thể này được sử dụng để xác định người thực hiện các thao tác điều khiển thiết bị trong hệ thống.

Tên cột	Kiểu dữ liệu	Khóa/ràng buộc	Mô tả
id	bigint	pk	Mã định danh duy nhất của người dùng.
username	varchar(50)	uk, not null	Tên đăng nhập của người dùng.
password_hash	varchar(25)	not null	Mật khẩu của người dùng được lưu dưới dạng mã hóa.
full_name	varchar(100)	not null	Họ và tên người dùng.
email	varchar(100)	not null	Địa chỉ email của người dùng.
created_at	datetime	not null	Thời điểm tạo tài khoản.

- Thực thể **Sensors**: Lưu thông tin các loại cảm biến được kết nối với hệ thống IoT. Mỗi cảm biến có mã định danh riêng, tên, loại cảm biến, đơn vị đo và MQTT topic dùng để nhận dữ liệu. Thông tin này giúp hệ thống phân biệt các loại dữ liệu như nhiệt độ, độ ẩm, ánh sáng.

Tên cột	Kiểu dữ liệu	Khóa/ràng buộc	Mô tả
id	bigint	pk	Mã định danh duy nhất của cảm biến.
code	varchar(50)	uk, not null	Mã định danh của cảm biến.
name	varchar(100)	not null	Tên cảm biến.
type	varchar(50)	not null	Loại cảm biến (temperature/ humidity/ light).
unit	varchar(20)	not null	Đơn vị đo của cảm biến.
mqtt_topic	varchar(25)	not null	MQTT topic được sử dụng để nhận dữ liệu từ cảm biến.
created_at	datetime	not null	Thời điểm cảm biến được thêm vào hệ

			thống.

- Thực thể **Sensor_data**: Lưu các giá trị được ESP32 thu thập từ cảm biến. Mỗi bản ghi bao gồm mã dữ liệu, mã cảm biến, giá trị đo được, thời điểm đo và thời điểm dữ liệu được lưu vào hệ thống. Đây là bảng chứa dữ liệu lịch sử phục vụ cho Dashboard, biểu đồ và chức năng tra cứu dữ liệu cảm biến.

Tên cột	Kiểu dữ liệu	Khóa/ràng buộc	Mô tả
id	bigint	pk	Mã định danh duy nhất của bản ghi dữ liệu.
sensor_id	bigint	fk, not null	Mã cảm biến tạo ra dữ liệu, tham chiếu đến bảng Sensors.
value	decimal(10,2)	not null	Giá trị đo được từ cảm biến.
measured_at	datetime	not null	Thời điểm cảm biến thực hiện phép đo.
created_at	datetime	not null	Thời điểm dữ liệu được lưu vào cơ sở dữ liệu.

- Thực thể **Devices**: Lưu thông tin các thiết bị có thể được điều khiển thông qua hệ thống. Mỗi thiết bị có mã thiết bị, tên, loại thiết bị, MQTT topic dùng để gửi lệnh điều khiển, topic nhận trạng thái và trạng thái hiện tại của thiết bị.

Tên cột	Kiểu dữ liệu	Khóa/ràng buộc	Mô tả
id	bigint	pk	Mã định danh duy nhất của thiết bị.
code	varchar(50)	uk, not null	Mã định danh của thiết bị.
name	varchar(100)	not null	Tên thiết bị.
type	varchar(50)	not null	Loại thiết bị.
command_topic	varchar(255)	not null	MQTT topic dùng để gửi lệnh điều khiển đến thiết bị.
status_topic	varchar(255)	not null	MQTT topic dùng để nhận trạng thái phản hồi từ thiết bị.

current_status	varchar(20)	not null	Trạng thái hiện tại của thiết bị (on/ off).
updated_at	datetime	not null	Thời điểm trạng thái thiết bị được cập nhật gần nhất.

- Thực thể **Action_history**: Lưu lịch sử các thao tác điều khiển thiết bị của người dùng. Mỗi bản ghi xác định người dùng nào đã thực hiện thao tác, thiết bị nào được điều khiển, hành động là bật hay tắt, trạng thái thiết bị và thời điểm gửi cũng như hoàn thành lệnh.

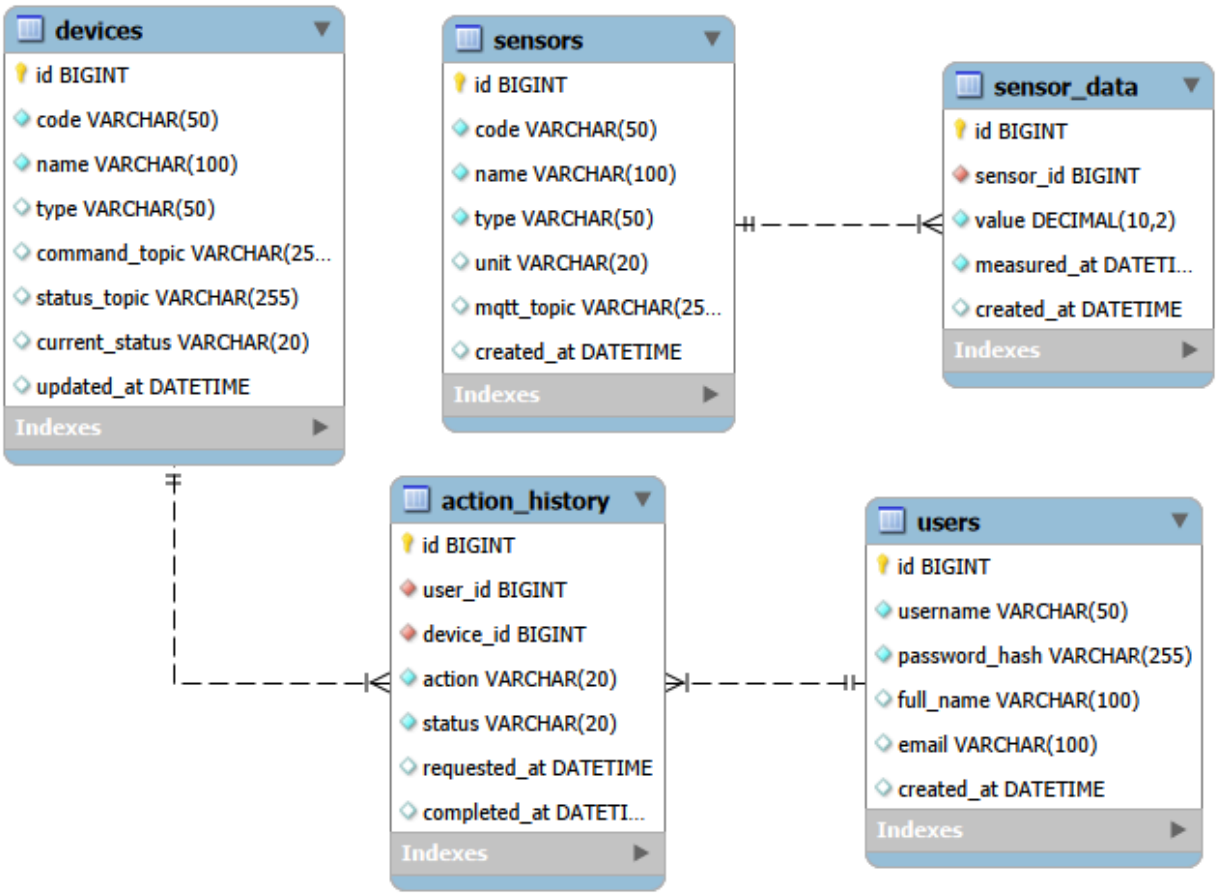
Tên cột	Kiểu dữ liệu	Khóa/ràng buộc	Mô tả
id	bigint	pk	Mã định danh duy nhất của bản ghi lịch sử.
user_id	bigint	fk, not null	Mã người dùng thực hiện thao tác, tham chiếu đến bảng Users.
device_id	bigint	fk, not null	Mã thiết bị được điều khiển, tham chiếu đến bảng Devices.
action	varchar(20)	not null	Hành động điều khiển thiết bị (on/ off).
status	varchar(20)	not null	Trạng thái xử lý của thao tác (pending/ success/ failed).
requested_at	datetime	not null	Thời điểm người dùng gửi yêu cầu điều khiển.
completed_at	datetime	null	Thời điểm thiết bị hoàn thành thao tác điều khiển.

b. Quan hệ giữa các thực thể

- Quan hệ giữa Sensors và Sensor_data: Quan hệ 1-N.
  Một cảm biến có thể có nhiều bản ghi dữ liệu cảm biến được ghi nhận theo thời gian, nhưng mỗi bản ghi Sensor_data chỉ thuộc về một cảm biến. Quan hệ được liên kết thông qua khóa ngoại sensor_id trong bảng Sensor_data.
- Quan hệ giữa Users và Action_history: Quan hệ 1-N.
  Một người dùng có thể thực hiện nhiều thao tác điều khiển thiết bị, nhưng mỗi bản ghi Action_history chỉ thuộc về một người dùng. Quan hệ được liên kết thông qua khóa ngoại user_id trong bảng Action_history.
- Quan hệ giữa Devices và Action_history: Quan hệ 1-N.

Một thiết bị có thể được điều khiển nhiều lần và mỗi lần điều khiển được lưu thành một bản ghi trong Action_history. Mỗi bản ghi Action_history chỉ liên quan đến một thiết bị. Quan hệ được liên kết thông qua khóa ngoại device_id trong bảng Action_history.

3.2.2. Sơ đồ quan hệ ERD



Hình 3.11. Sơ đồ thực thể ERD

3.3. Mô tả API

3.3.1. API lấy dữ liệu cảm biến mới nhất

- Method: GET
- Endpoint: /api/sensor-data/latest
- Mục đích: Lấy dữ liệu cảm biến mới nhất để hiển thị trên Dashboard
- Request URL: http://localhost:8080/api/sensor-data/latest
- Request body: Không có
- Response body (200 OK):

```
[

    {
```

```
        "sensor_id": 1,
        "sensor_code": "TEMP01",
        "sensor_name": "Temperature Sensor",
        "sensor_type": "temperature",
        "unit": "°C",
        "value": 31.30,
        "measured_at": "2026-10-13T15:14:38"
    }
]
```


3.3.2. API lấy lịch sử dữ liệu cảm biến

- Method: GET
- Endpoint: /api/sensor-data/history
- Mục đích: Lấy dữ liệu cảm biến đã được lưu để hiển thị biểu đồ hoặc lịch sử đo.
- Request URL: http://localhost:8080/api/sensor-data/history
- Request body: Không có
- Response body (200 OK):


```
[
    {
        "measured_at": "2026-10-13T15:19:38",
        "temperature": 31.30,
        "light": 2208.00,
        "humidity": 68.00
    },
    {
        "measured_at": "2026-10-13T15:19:40",
        "temperature": 31.30,
        "light": 2236.00,
```

```
        "humidity": 68.00
    }
]
```


3.3.3. API lấy danh sách dữ liệu cảm biến

- Method: GET
- Endpoint: /api/sensor-data
- Mục đích: Lấy danh sách dữ liệu cảm biến để hiển thị tại trang Datasensor
- Request URL: http://localhost:8080/api/sensor-data
- Request body: Không có
- Response body (200 OK):


```
[
    {
        "id": 27665,
        "sensor_type": "humidity",
        "sensor_name": "Humidity Sensor",
        "value": 68.00,
        "unit": "%",
        "measured_at": "2026-10-13T15:20:13"
    },
    {
        "id": 27664,
        "sensor_type": "light",
        "sensor_name": "Light Sensor",
        "value": 2254.00,
        "unit": "Lux",
        "measured_at": "2026-10-13T15:20:13"
    },
```

```
    {
        "id": 27663,
        "sensor_type": "temperature",
        "sensor_name": "Temperature Sensor",
        "value": 31.30,
        "unit": "°C",
        "measured_at": "2026-10-13T15:20:13"
    }
]
```


3.3.4. API lấy trạng thái thiết bị

- Method: GET
- Endpoint: /api/devices/status
- Mục đích: Lấy trạng thái hiện tại của các thiết bị để hiển thị trên Dashboard
- Request URL: http://localhost:8080/api/devices/status
- Request body: Không có
- Response body (200 OK):


```
[
    {
        "id": 1,
        "name": "Light",
        "current_status": "OFF"
    },
    {
        "id": 2,
        "name": "Air Conditioner",
        "current_status": "OFF"
    },
```

```
    {
        "id": 3,
        "name": "Fan",
        "current_status": "OFF"
    }
]
```


3.3.5. API điều khiển thiết bị

- Method: POST
- Endpoint: /api/device/control
- Mục đích: Gửi yêu cầu bật hoặc tắt thiết bị từ giao diện Web đến Backend
- Request URL: http://localhost:8080/api/device/control
- Request body:


```
{
  "led": "led2",
  "action": "on"
}
```


- Response body (200 OK):


```
{
    "status": "Pending",
    "led": "led2",
    "action": "On",
    "message": "Da gui lenh MQTT"
}
```


3.3.6. API lấy lịch sử hành động

- Method: GET
- Endpoint: /api/history
- Mục đích: Lấy lịch sử bật/tắt thiết bị đã được ghi nhận trong hệ thống
- Request URL: http://localhost:8080/api/history

- Request body: Không có
- Response body (200 OK):


```
[
    {
        "id": 69,
        "device_name": "Air Conditioner",
        "action": "Off",
        "device_status": "Off",
        "performed_by": "thuydungvu",
        "time": "2026-10-13T15:20:02"
    },
    {
        "id": 68,
        "device_name": "Air Conditioner",
        "action": "On",
        "device_status": "On",
        "performed_by": "thuydungvu",
        "time": "2026-10-13T15:19:48"
    }
]
```


3.3.7. MQTT API - Lấy dữ liệu cảm biến mới nhất

- Topic: esp32/sensor-data
- Mục đích: Gửi dữ liệu cảm biến từ ESP32 lên MQTT Broker để Backend nhận dữ liệu, lưu vào MySQL và cập nhật giao diện Frontend.
- Publisher: ESP32
- Subscriber: Backend (Spring Boot)
- Message:

{
 "temperature": 30.7,
 "humidity": 52.1,
 "light": 600,
 "time": "13/10/2026 14:30:00"
}

3.3.8. MQTT API - Điều khiển thiết bị

- Topic: esp32/device-control
- Mục đích: Truyền lệnh điều khiển từ Backend xuống ESP32 để bật hoặc tắt các thiết bị thông qua giao diện Dashboard.
- Publisher: Backend (Spring Boot)
- Subscriber: ESP32
- Message:

{\"led1\":\"on\"}

3.3.9. MQTT API - Lấy trạng thái thiết bị

- Topic: esp32/device-status
- Mục đích: Gửi trạng thái thực tế của thiết bị từ ESP32 về Backend sau khi xử lý lệnh điều khiển, giúp Backend cập nhật trạng thái thiết bị trong MySQL và thông báo cho Frontend qua WebSocket.
- Publisher: ESP32
- Subscriber: Backend (Spring Boot)
- Message:

{

 "id": 1,

 "action": "ON",

 "device_status": "SUCCESS",

 "time": "2026-10-11T15:28:43"

}

3.3.10. WebSocket API - Lấy dữ liệu cảm biến mới nhất

- Mục đích: Đẩy dữ liệu cảm biến mới nhất từ Backend xuống Frontend ngay khi Backend nhận được dữ liệu từ ESP32 qua MQTT, giúp Dashboard cập nhật dữ liệu nhiệt độ, độ ẩm và ánh sáng theo thời gian thực mà không cần Frontend gọi API liên tục.
- Endpoint kết nối: ws://localhost:8080/ws
- Channel subscribe: /topic/sensors

- Publisher: Backend
- Subscriber: Frontend
- Message:


```
{
 "temperature": 30.7,
 "humidity": 52.1,
 "light": 600,
 "timestamp": "2026-10-13T14:30:00"
}
```


3.3.11. WebSocket API - Lấy trạng thái thiết bị

- Mục đích: Đẩy trạng thái thiết bị mới nhất từ Backend xuống Frontend ngay khi Backend nhận được trạng thái từ ESP32 qua MQTT, giúp Dashboard cập nhật trạng thái bật/ tắt của thiết bị theo thời gian thực mà không cần Frontend gọi API liên tục.
- Endpoint kết nối: ws://localhost:8080/ws
- Channel subscribe: /topic/device-status
- Publisher: Backend
- Subscriber: Frontend
- Message:


```
{
 "device": "led1",
 "status": "On"
}
```

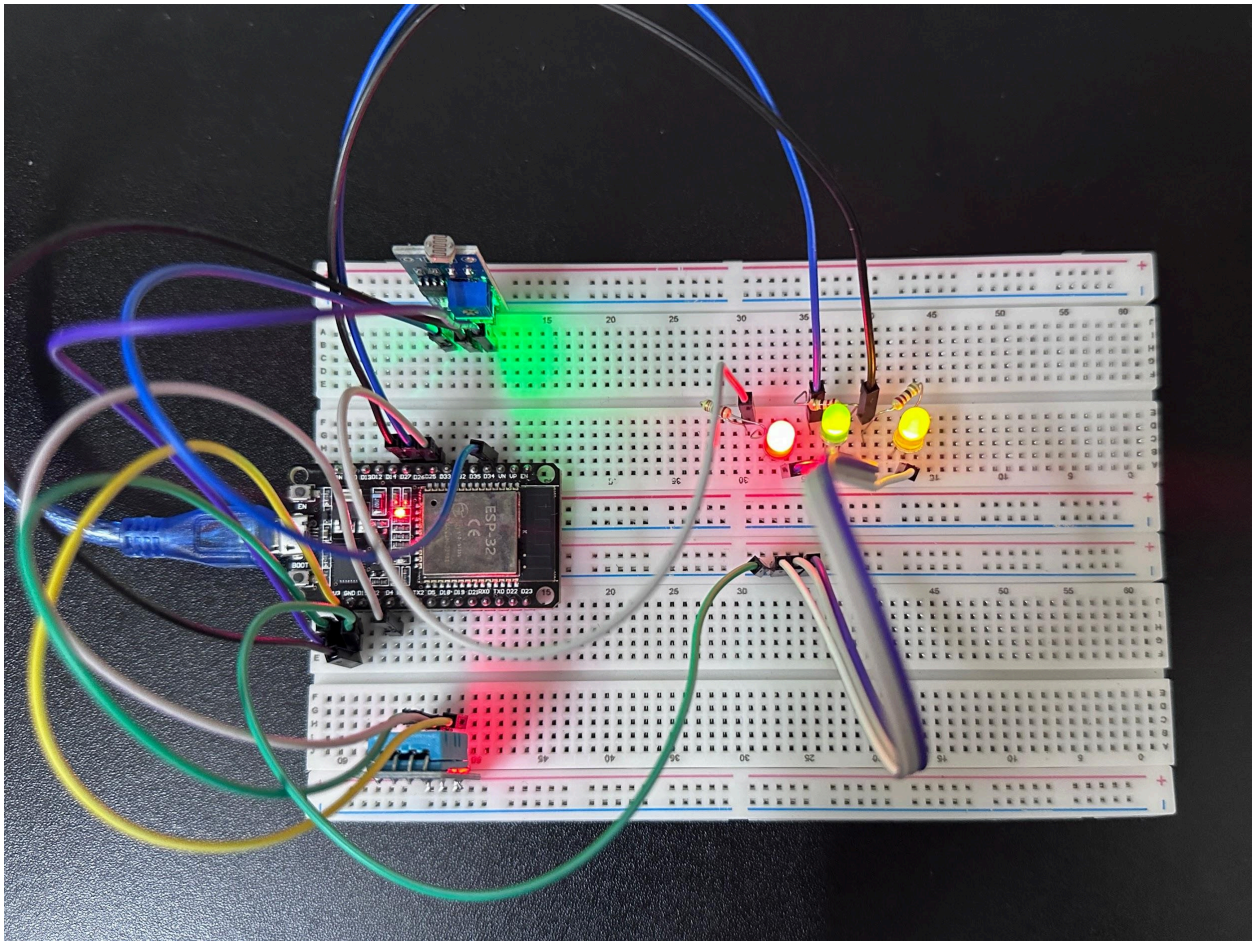

3.4. Thiết kế phần cứng

3.4.1. Sơ đồ chân kết nối

Linh kiện	Chân linh kiện	Chân ESP32 DEVKIT V1	Ghi chú
LDR	A0	D34 (GPIO 34)	
	VCC	3V3	
	GND	GND	
DHT11	DATA	D4 (GPIO 4)	
	VCC	3V3	
	GND	GND	
LED 1	Anode (+)	D25 (GPIO 25)	Đi qua điện trở 220Ω

	Cathode (-)	GND	
LED 2	Anode (+)	D26 (GPIO 26)	Đi qua điện trở 220Ω
	Cathode (-)	GND	
LED 3	Anode (+)	D27 (GPIO 27)	Đi qua điện trở 220Ω
	Cathode (-)	GND	

3.4.2. Đấu nối mạch



Hình 3.12. Mạch đã đấu nối

Chương IV. Kết quả và đánh giá thực nghiệm

4.1. Cấu trúc thư mục dự án

```
IoT/
├── ESP32/
│   └── ESP32.ino
│
├── IoT-Frontend/
│   ├── assets/
│   │   ├── icons/
│   │   │   ├── air-conditioner.svg
│   │   │   ├── fan.svg
│   │   │   └── light.svg
│   │   └── images/
│   │       └── anh.jpg
│   ├── css/
│   │   ├── common.css
│   │   ├── dashboard.css
│   │   ├── datasensor.css
│   │   ├── history.css
│   │   └── profile.css
│   ├── js/
│   │   ├── common/
│   │   │   └── pagination.js
│   │   ├── dashboard/
│   │   │   ├── dashboard.js
│   │   │   ├── device-control.js
│   │   │   └── sensor-chart.js
│   │   ├── datasensor/
│   │   │   └── datasensor.js
│   │   ├── history/
│   │   │   └── history.js
│   │   ├── profile/
│   │   │   └── profile.js
│   │   └── services/
│   │       ├── api.js
│   │       ├── mock-data.js
│   │       └── websocket.js
│   └── pages/
│       ├── dashboard.html
│       ├── datasensor.html
│       └── history.html
```

```
        └── profile.html
│
└── iot-system/
    ├── pom.xml
    └── src/
        └── main/
            ├── java/com/iot/iotsystem/
            │   ├── IotSystemApplication.java
            │   ├── config/
            │   │   ├── CorsConfig.java
            │   │   ├── MqttConfig.java
            │   │   └── WebSocketConfig.java
            │   ├── controller/
            │   │   ├── DeviceController.java
            │   │   ├── DeviceStateController.java
            │   │   ├── DeviceStatusController.java
            │   │   ├── HistoryController.java
            │   │   ├── SensorController.java
            │   │   ├── SensorDataController.java
            │   │   └── SensorRealtimeController.java
            │   ├── mqtt/
            │   │   ├── MqttSensorListener.java
            │   │   └── MqttStatusSubscriber.java
            │   └── service/
            │       └── MqttService.java
            └── resources/
                └── application.properties
```

4.2. Kết quả triển khai phần cứng và MQTT

4.2.1. Phần cứng

Phần cứng của hệ thống đã được chế tạo và lắp ráp hoàn chỉnh trên bo mạch cắm (breadboard) thử nghiệm với độ tin cậy và thẩm mỹ cao:

- Bộ vi điều khiển: ESP32 DevKit V1 được nạp mã nguồn C++ phát triển trên nền tảng Arduino IDE.
- Khối cảm biến: Cảm biến DHT11 và quang trở LDR hoạt động ổn định, đọc dữ liệu định kỳ chuẩn xác.
- Khối hiển thị & chấp hành: 3 đèn LED đỏ, vàng, xanh phản ứng tức thì khi nhận được tín hiệu kích hoạt từ các chân GPIO.
- Kết nối mạng & Thời gian thực: ESP32 tự động quét và kết nối thành công vào mạng Wi-Fi, kết nối đồng bộ thành công với máy chủ và gán nhãn thời gian thực chuẩn GMT+7 vào từng gói tin.

4.2.2. MQTT Broker

- Broker Mosquitto: Chạy ổn định ở cổng 1883, tiếp nhận thông lượng gói tin đều đặn mỗi 2 giây từ phần cứng và cho phép Spring Boot Client subscribe lấy tin tức thời.

```
C:\Users\vuthu>"C:\Program Files\mosquitto\mosquitto.exe" -c "D:\mosquitto\mosquitto.conf" -v
1788841062: mosquitto version 2.1.2 starting
1788841062: Config loaded from D:\mosquitto\mosquitto.conf.
1788841062: Bridge support available.
1788841062: Persistence support available.
1788841062: TLS support available.
1788841062: TLS-PSK support available.
1788841062: Websockets support available.
1788841062: Plugin builtin-security has registered to receive 'basic-auth' events.
1788841062: Opening ipv4 listen socket on port 1883.
1788841062: mosquitto version 2.1.2 running
```

Hình 4.1. Khởi chạy MQTT Broker

- Kiểm thử giả lập lắng nghe dữ liệu cảm biến: Dùng lệnh mosquitto_sub trên Terminal cmd để đăng ký lắng nghe topic esp32/sensor-data.

```
C:\Users\vuthu>mosquitto_sub -h 192.168.1.5 -u "thuydungvu" -P "123456" -p 1883 -t "esp32/sensor-data" -v
esp32/sensor-data {"time":"08/09/2026 22:06:06","temperature":32.80,"humidity":76.00,"light":1331}
esp32/sensor-data {"time":"08/09/2026 22:06:08","temperature":32.80,"humidity":76.00,"light":1337}
esp32/sensor-data {"time":"08/09/2026 22:06:10","temperature":32.80,"humidity":76.00,"light":1328}
esp32/sensor-data {"time":"08/09/2026 22:06:12","temperature":32.80,"humidity":76.00,"light":1328}
esp32/sensor-data {"time":"08/09/2026 22:06:14","temperature":32.80,"humidity":76.00,"light":1330}
esp32/sensor-data {"time":"08/09/2026 22:06:16","temperature":32.80,"humidity":76.00,"light":1311}
```

Hình 4.2. Kết quả thu thập dữ liệu cảm biến

- Kiểm thử giả lập điều khiển thiết bị: Sử dụng lệnh mosquitto_pub để publish các message điều khiển lên topic esp32/device-control, từ đó gửi lệnh bật/ tắt LED đến ESP32 thông qua MQTT Broker.

```
C:\Users\vuthu>mosquitto_pub -h 192.168.1.5 -u "thuydungvu" -P "123456" -p 1883 -t "esp32/device-control" -m "{\"led1\":\"on\",\"led2
\":\"on\",\"led3\":\"on\"}"
```

Hình 4.3. Lệnh điều khiển bật 3 đèn LED

Kết quả: ESP32 nhận được lệnh ON, chân D25/ D26/ D27 được thiết lập ở mức logic HIGH tạo dòng điện lần lượt đi qua điện trở 220Ω và LED 1/ 2/ 3 xuống GND làm đèn sáng.

4.3. Kết quả triển khai Backend và CSDL

4.3.1. Backend

- Khởi động thành công trên cổng 8080, kết nối với MySQL để quản lý dữ liệu hệ thống IoT.
- Tích hợp các thành phần Backend để tiếp nhận dữ liệu cảm biến, xử lý lệnh điều khiển thiết bị và lưu trữ lịch sử hoạt động.
- Tích hợp MQTT: Backend subscribe topic esp32/sensor-data để nhận dữ liệu nhiệt độ, độ ẩm, ánh sáng từ ESP32; publish lệnh bật/tắt thiết bị lên esp32/device-control và subscribe esp32/device-status để nhận trạng thái phản hồi từ ESP32.
- Quản lý dữ liệu cảm biến: Lưu dữ liệu cảm biến vào MySQL, cung cấp REST API để truy vấn dữ liệu mới nhất và lịch sử đo, phục vụ hiển thị, tìm kiếm và lọc dữ liệu trên giao diện Datasensor.

- Quản lý thiết bị: Xử lý yêu cầu điều khiển từ Frontend thông qua API POST /api/device/control, gửi lệnh qua MQTT và cập nhật trạng thái thiết bị khi nhận được phản hồi từ ESP32.
- Quản lý lịch sử hoạt động: Ghi nhận lịch sử bật/tắt thiết bị vào cơ sở dữ liệu, hỗ trợ Frontend truy vấn và theo dõi các thao tác điều khiển.
- Tích hợp WebSocket STOMP: Backend phát thông báo theo thời gian thực qua các channel /topic/device-status và /topic/sensors, giúp Dashboard cập nhật trạng thái thiết bị và dữ liệu cảm biến mà không cần tải lại trang.
- Đồng bộ dữ liệu giữa các thành phần: Kết hợp REST API, MQTT và WebSocket để đảm bảo Frontend có thể truy vấn dữ liệu đã lưu, đồng thời nhận các cập nhật mới từ hệ thống IoT theo thời gian thực.

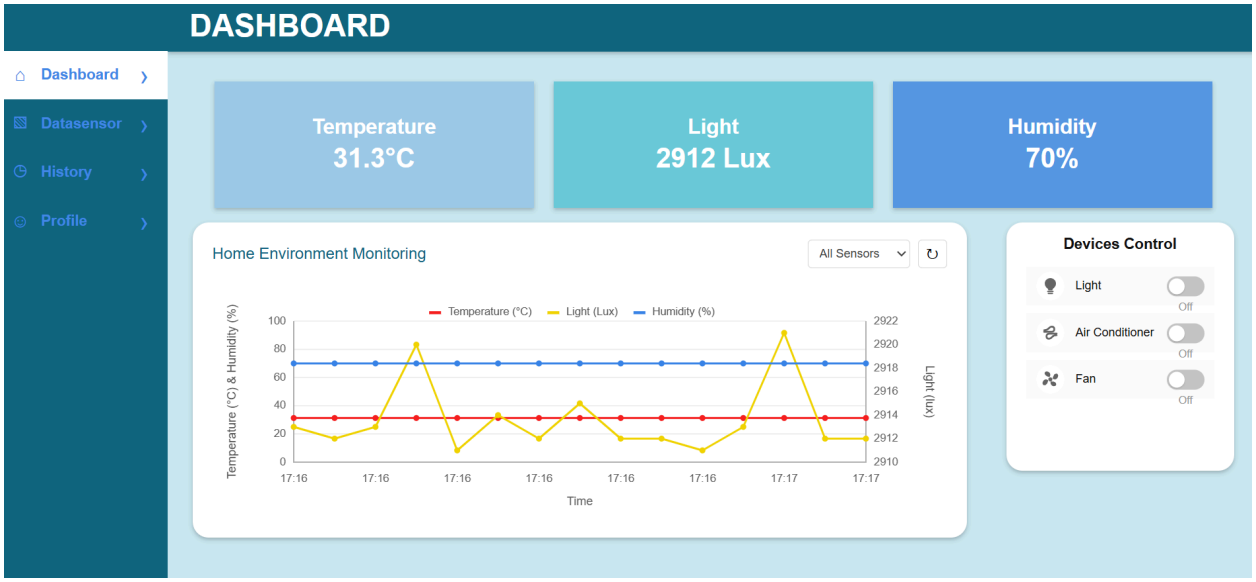
4.3.2. Cơ sở dữ liệu

- Xây dựng và triển khai cơ sở dữ liệu iot_system để lưu trữ dữ liệu phục vụ hệ thống giám sát và điều khiển thiết bị IoT.
- Quản lý tài khoản người dùng: Bảng users lưu thông tin tài khoản, tên người dùng, họ tên, email và thông tin xác thực phục vụ chức năng đăng nhập.
- Quản lý cảm biến: Bảng sensors lưu thông tin cảm biến, bao gồm mã cảm biến, tên, loại, đơn vị đo và MQTT topic tương ứng.
- Lưu trữ dữ liệu cảm biến: Bảng sensor_data lưu các giá trị nhiệt độ, độ ẩm và ánh sáng cùng thời điểm đo, phục vụ hiển thị dữ liệu mới nhất, biểu đồ và tra cứu lịch sử.
- Quản lý thiết bị: Bảng devices lưu thông tin thiết bị điều khiển, trạng thái bật/tắt và các topic MQTT dùng để gửi lệnh, nhận phản hồi.
- Lưu trữ lịch sử điều khiển: Bảng action_history ghi nhận các hoạt động điều khiển và trạng thái thiết bị, hỗ trợ theo dõi lịch sử thao tác.
- Liên kết dữ liệu: Thiết kế quan hệ giữa dữ liệu cảm biến, thông tin cảm biến, thiết bị và lịch sử điều khiển nhằm đảm bảo tính nhất quán và hỗ trợ truy vấn dữ liệu.
- Tích hợp Backend: Spring Boot kết nối MySQL thông qua cấu hình datasource và sử dụng các câu truy vấn để lưu, cập nhật, lấy dữ liệu cảm biến, trạng thái thiết bị và lịch sử hoạt động.
- Đảm bảo tính liên tục của dữ liệu: Trạng thái thiết bị và dữ liệu cảm biến được lưu trong cơ sở dữ liệu, cho phép Frontend truy vấn lại dữ liệu sau khi tải lại trang thay vì chỉ phụ thuộc vào dữ liệu đang hiển thị trên giao diện.

4.4. Kết quả triển khai Frontend

a. Trang Dashboard

Trang Dashboard hiển thị trực quan các thông số thu thập từ cảm biến nhiệt độ, độ ẩm và ánh sáng. Dữ liệu được biểu diễn bằng biểu đồ động cho thấy sự thay đổi giá trị theo thời gian, đồng thời cung cấp các nút thao tác để người dùng dễ dàng điều khiển thiết bị.



Hình 4.4. Trang Dashboard

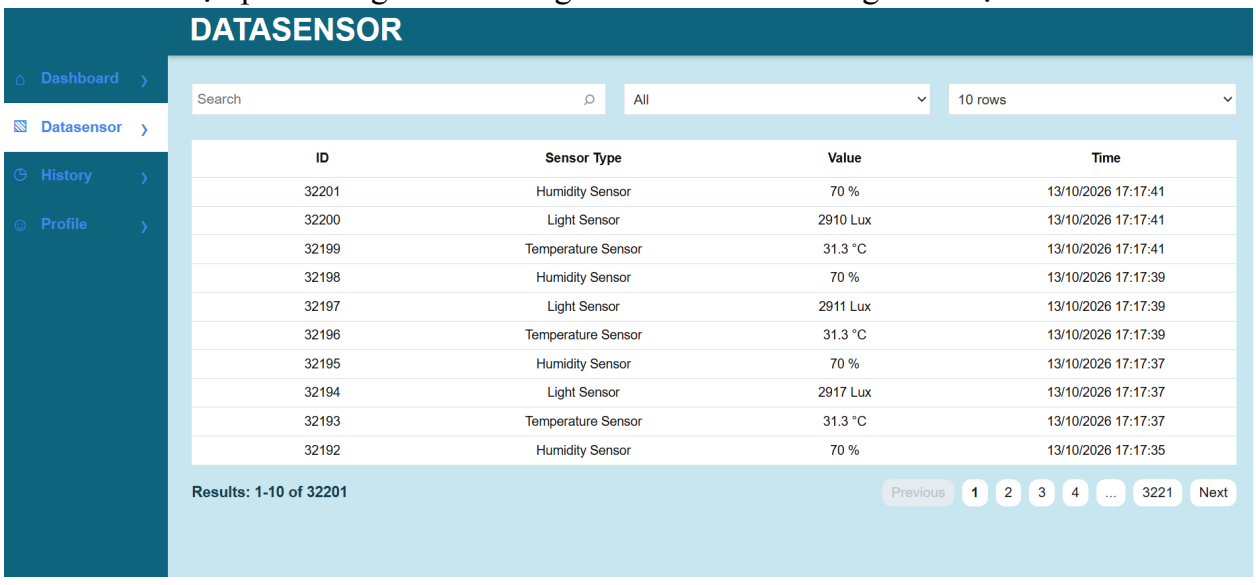
b. Trang Datasensor

Trang Dữ liệu cảm biến (Datasensor) lưu trữ lại các bản ghi dữ liệu thu được từ các cảm biến theo thời gian thực.

Bảng dữ liệu cảm biến (Datasensor) gồm 4 cột chính:

- ID: Số thứ tự bản ghi
- Sensor Type: Tên loại cảm biến
- Value: Giá trị của cảm biến
- Time: Thời điểm ghi nhận dữ liệu

Trang Datasensor có một khung tìm kiếm để người dùng tìm kiếm thông tin mong muốn, một khung lựa chọn loại bộ lọc, một khung chọn số lượng bản ghi hiển thị. Trang Datasensor được phân trang và mỗi trang chứa tối đa 100 bản ghi dữ liệu.



Hình 4.5. Trang Datasensor

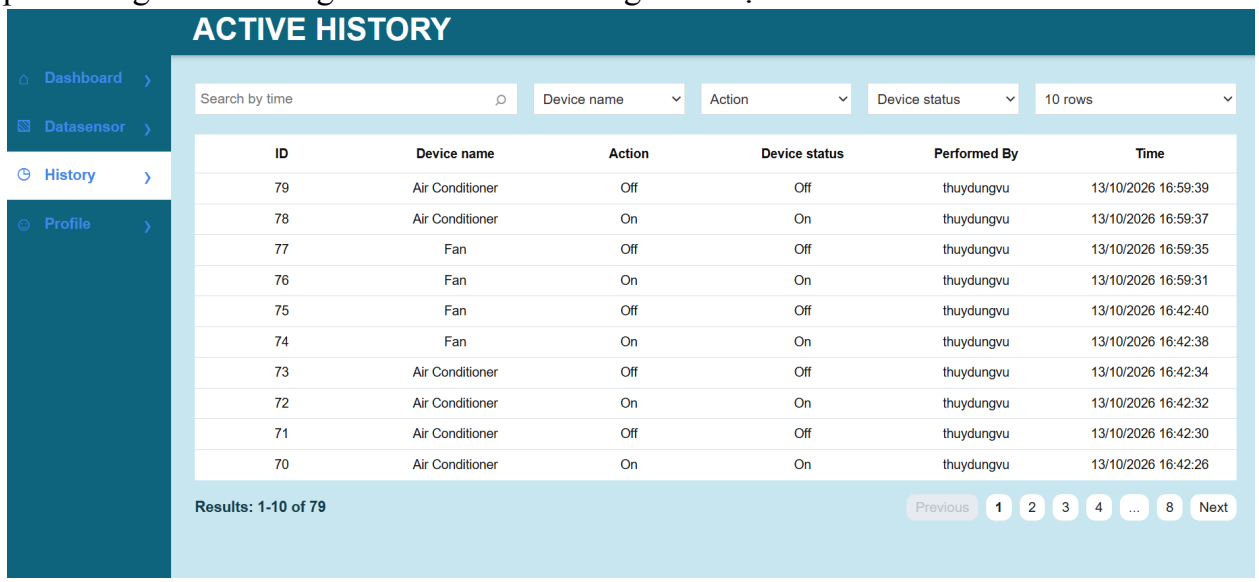
c. Trang Dashboard

Trang lịch sử hoạt động (Active History) lưu trữ lại các bản ghi lịch sử hoạt động bật tắt

của các thiết bị theo thời gian thực.
Bảng lịch sử hoạt động (Active History) gồm 6 cột chính:

- ID: Số thứ tự bản ghi.
- Device name: Tên thiết bị được điều khiển (quạt, đèn, điều hoà).
- Action: Hành động bật/ tắt của người nhấn.
- Device status: Trạng thái tại một thời điểm của thiết bị (bật/ tắt).
- Performed by: Người thực hiện hành động điều khiển thiết bị.
- Time: Thời điểm ghi nhận thao tác điều khiển.

Trang Active History có một khung tìm kiếm theo thời gian, một khung lựa chọn loại cảm biến cần tìm, một khung lựa chọn loại hành động (bật/ tắt), một khung lựa chọn loại trạng thái thiết bị, một khung chọn số lượng bản ghi hiển thị. Trang Active History được phân trang và mỗi trang chứa tối đa 100 bản ghi dữ liệu.



Hình 4.6. Trang Active History

d. Trang Dashboard

Trang Profile hiển thị các thông tin cá nhân và các đường dẫn đến Github, Figma, Postman và tài liệu PDF mô tả dự án.



Hình 4.7. Trang Profile

4.5. Triển khai hệ thống

Kết quả vận hành thực tế:

- Thu thập dữ liệu: ESP32 đọc dữ liệu nhiệt độ, độ ẩm và ánh sáng, sau đó gửi lên Backend thông qua MQTT để lưu vào MySQL.
- Điều khiển thiết bị: Người dùng có thể bật/ tắt các thiết bị LED từ Dashboard. Backend gửi lệnh qua MQTT và nhận trạng thái phản hồi từ ESP32.
- Cập nhật thời gian thực: WebSocket STOMP hỗ trợ gửi dữ liệu cảm biến và trạng thái thiết bị đến Frontend mà không cần tải lại trang.
- Lưu trữ và tra cứu: Dữ liệu cảm biến và lịch sử điều khiển được lưu trong MySQL, phục vụ việc hiển thị và truy vấn trên các trang Datasensor và History.
- Khả năng vận hành: Hệ thống đã triển khai các chức năng giám sát cảm biến, điều khiển thiết bị và ghi nhận lịch sử hoạt động trong môi trường cục bộ.

Chương V. Kết luận

5.1. Kết quả đạt được

Sau quá trình nghiên cứu, thiết kế và triển khai, đề tài **"Xây dựng hệ thống IoT giám sát và điều khiển thiết bị bằng giao diện Web"** đã hoàn thành các chức năng chính, kết hợp giữa phần cứng ESP32, giao thức MQTT, Backend Spring Boot, cơ sở dữ liệu MySQL và giao diện Web.

Về phần cứng, hệ thống sử dụng ESP32 DevKit V1 để thu thập dữ liệu từ cảm biến nhiệt độ, độ ẩm DHT11 và cảm biến ánh sáng LDR. Đồng thời, ESP32 có khả năng tiếp nhận lệnh điều khiển và thực hiện bật/tắt các thiết bị LED được kết nối với các chân GPIO tương ứng.

Về giao tiếp dữ liệu, hệ thống sử dụng MQTT với ba topic chính gồm esp32/sensor-data, esp32/device-control và esp32/device-status. Cơ chế publish/subscribe giúp trao đổi dữ liệu cảm biến, truyền lệnh điều khiển và phản hồi trạng thái thiết bị giữa ESP32 và Backend.

Về Backend, Spring Boot đảm nhiệm việc tiếp nhận, xử lý dữ liệu MQTT, cung cấp REST API phục vụ giao diện Web và cập nhật thông tin trạng thái thiết bị. MySQL được sử dụng để lưu trữ dữ liệu cảm biến, thông tin thiết bị, tài khoản người dùng và lịch sử điều khiển, hỗ trợ truy vấn dữ liệu khi cần thiết.

Về giao diện, hệ thống được xây dựng bằng HTML, CSS và JavaScript với các trang Dashboard, Datasensor và History. Dashboard cho phép theo dõi các thông số cảm biến và điều khiển thiết bị; Datasensor hỗ trợ xem dữ liệu đo; History phục vụ tra cứu lịch sử hoạt động. Việc tích hợp WebSocket STOMP giúp cập nhật dữ liệu cảm biến và trạng thái thiết bị theo thời gian thực mà không cần tải lại trang khi có thông báo mới.

Nhìn chung, đề tài đã xây dựng được quy trình hoạt động cơ bản của một hệ thống IoT hoàn chỉnh, từ thu thập dữ liệu tại thiết bị phần cứng, truyền dữ liệu qua mạng, xử lý và lưu trữ tại máy chủ đến hiển thị thông tin và điều khiển thông qua giao diện Web. Kết quả này đáp ứng các mục tiêu chính của đề tài và tạo nền tảng cho việc mở rộng hệ thống trong tương lai.

5.2. Hạn chế

Bên cạnh những kết quả đạt được, hệ thống vẫn còn một số hạn chế cần được khắc phục.

Thứ nhất, hệ thống hiện sử dụng các cảm biến và thiết bị phần cứng cơ bản, chủ yếu phục vụ mục đích thử nghiệm. Phạm vi giám sát còn hạn chế, chưa đáp ứng được các yêu cầu phức tạp của môi trường triển khai thực tế.

Thứ hai, hệ thống phụ thuộc vào kết nối Wi-Fi, MQTT Broker và máy chủ Backend. Khi

một trong các thành phần này gặp sự cố, quá trình truyền dữ liệu hoặc điều khiển thiết bị có thể bị gián đoạn. Việc khôi phục kết nối và xử lý các tình huống mất mạng cần được hoàn thiện hơn.

Thứ ba, hiệu năng của hệ thống khi số lượng bản ghi cảm biến tăng lớn hoặc có nhiều thiết bị gửi dữ liệu đồng thời chưa được đánh giá đầy đủ thông qua các bài kiểm thử tải chuyên biệt. Việc truy vấn lịch sử, tìm kiếm và phân trang có thể cần tối ưu thêm khi quy mô dữ liệu tăng lên.

Thứ tư, các cơ chế bảo mật cần tiếp tục được tăng cường, bao gồm xác thực người dùng, phân quyền truy cập, bảo vệ API và bảo mật kết nối MQTT. Đây là những yếu tố quan trọng nếu hệ thống được triển khai trên môi trường mạng thực tế.

Cuối cùng, các chức năng phân tích dữ liệu và cảnh báo hiện còn ở mức cơ bản. Hệ thống chưa phát triển đầy đủ các cơ chế cảnh báo tự động, thống kê chuyên sâu hoặc phân tích xu hướng dựa trên dữ liệu cảm biến trong thời gian dài.

5.3. Hướng phát triển

Trong thời gian tới, hệ thống có thể được phát triển theo các hướng sau:

- Mở rộng phần cứng: Bổ sung các loại cảm biến như cảm biến chuyển động, cảm biến khí gas hoặc cảm biến chất lượng không khí để mở rộng khả năng giám sát môi trường.
- Nâng cao khả năng giám sát: Phát triển thêm biểu đồ thống kê theo thời gian, báo cáo theo ngày/tháng và chức năng xuất dữ liệu nhằm hỗ trợ người dùng phân tích tình trạng môi trường.
- Xây dựng hệ thống cảnh báo: Thiết lập ngưỡng cho nhiệt độ, độ ẩm và ánh sáng; khi các thông số vượt ngưỡng, hệ thống có thể gửi thông báo tới giao diện hoặc các kênh thông báo phù hợp.
- Tăng cường bảo mật: Hoàn thiện cơ chế đăng nhập, phân quyền người dùng, xác thực MQTT, bảo vệ REST API và sử dụng kết nối mã hóa khi triển khai trên mạng thực tế.
- Tối ưu hiệu năng: Cải thiện truy vấn MySQL, bổ sung chỉ mục phù hợp, tối ưu phân trang và xây dựng cơ chế xử lý dữ liệu hiệu quả khi số lượng thiết bị và bản ghi tăng lên.
- Nâng cao độ ổn định: Bổ sung cơ chế tự động kết nối lại khi mất Wi-Fi hoặc MQTT, phát hiện thiết bị ngoại tuyến và thông báo tình trạng kết nối cho người dùng.
- Triển khai trên nền tảng máy chủ: Đưa Backend, cơ sở dữ liệu và MQTT Broker lên máy chủ có thể truy cập qua mạng, cho phép giám sát và điều khiển thiết bị từ xa thay vì chỉ hoạt động trong môi trường cục bộ.
- Phát triển ứng dụng mở rộng: Xem xét xây dựng giao diện trên thiết bị di động hoặc tích hợp các phương pháp phân tích dữ liệu để hỗ trợ phát hiện bất thường và dự đoán xu hướng môi trường.

Tóm lại, đề tài đã xây dựng được nền tảng giám sát và điều khiển thiết bị IoT thông qua giao diện Web, vận dụng các công nghệ MQTT, Spring Boot, MySQL và WebSocket. Mặc dù vẫn còn những hạn chế về phạm vi chức năng, bảo mật và khả năng mở rộng, hệ thống có tiềm năng phát triển thành một giải pháp giám sát và điều khiển linh hoạt hơn, đáp ứng tốt hơn nhu cầu sử dụng trong thực tế.